

CS 31204: Computer Networks – Application Layer

Department of Computer Science
and Engineering



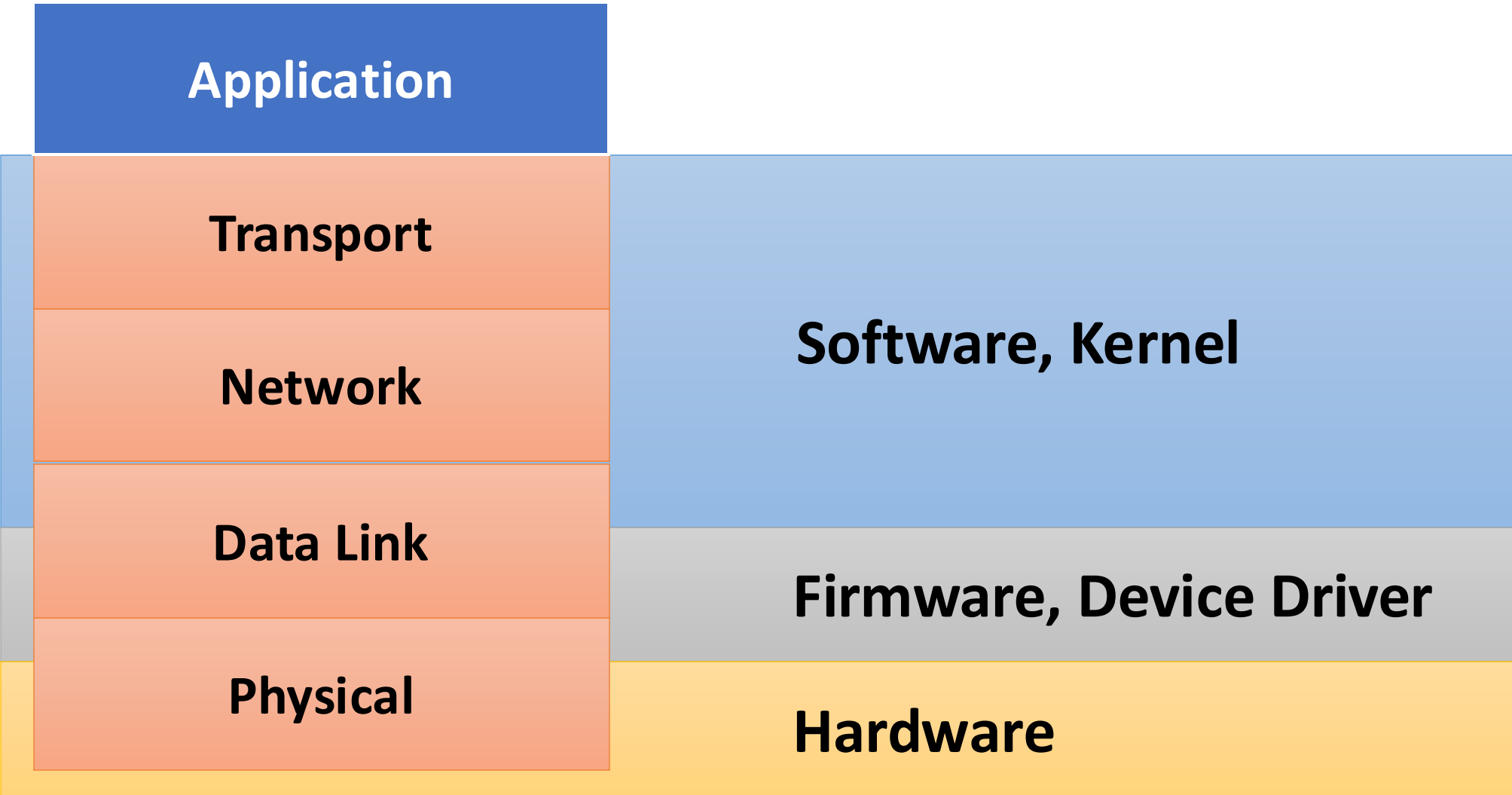
INDIAN INSTITUTE OF TECHNOLOGY
KHARAGPUR



Abhijnan Chakraborty
abhijnan@cse.iitkgp.ac.in

Sandip Chakraborty
sandipc@cse.iitkgp.ac.in

Protocol Stack Implementation in a Host



Some Network Applications

- social networking
- Web
- text messaging
- e-mail
- multi-user network games
- streaming stored video (YouTube, Hulu, Netflix)
- P2P file sharing
- voice over IP (e.g., Skype)
- real-time video conferencing (e.g., Zoom)
- Internet search
- remote login
- ...

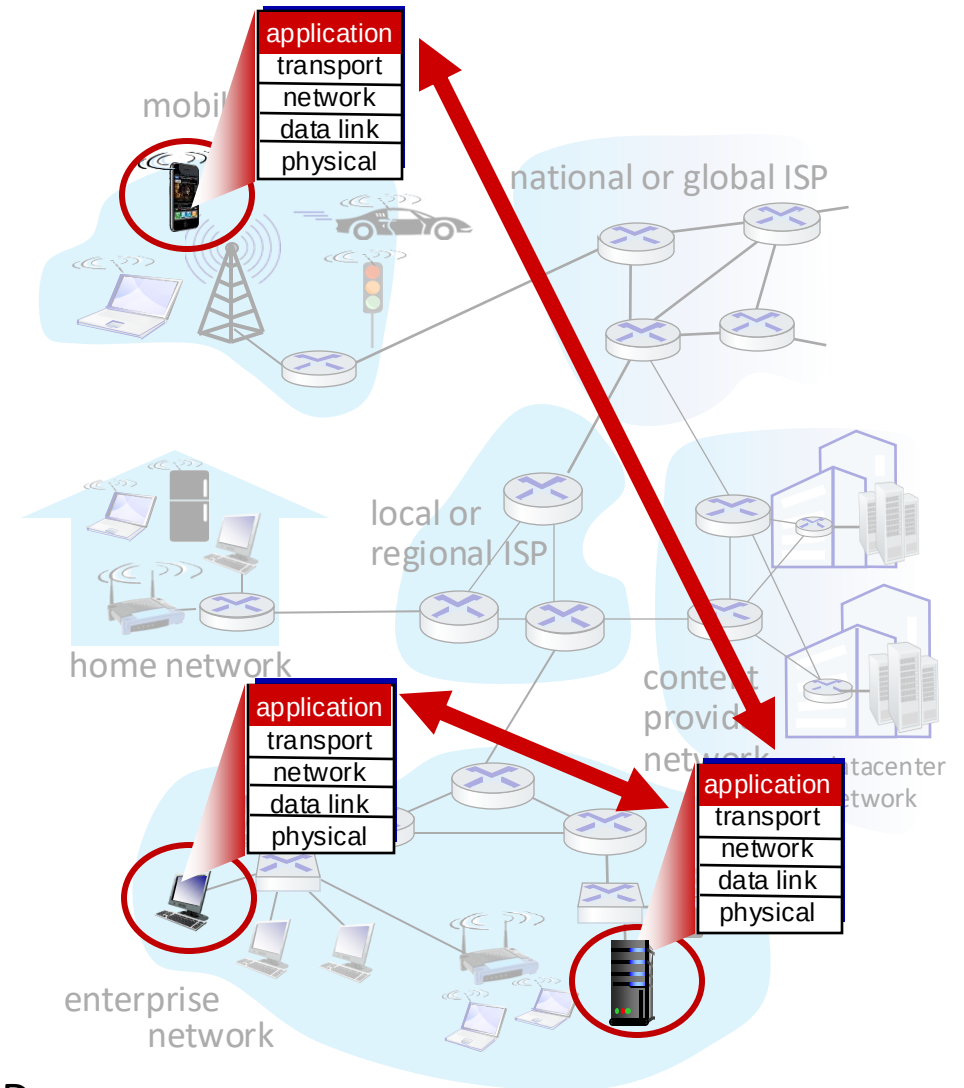
Creating a Network App

write programs that:

- run on (different) end systems
- communicate over network
- e.g., web server software communicates with browser software

no need to write software for network-core devices

- network-core devices do not run user applications
- applications on end systems allows for rapid app development, propagation



Source: Computer Networking: A Top-Down Approach (8th Ed) by Jim Kurose, Keith Ross

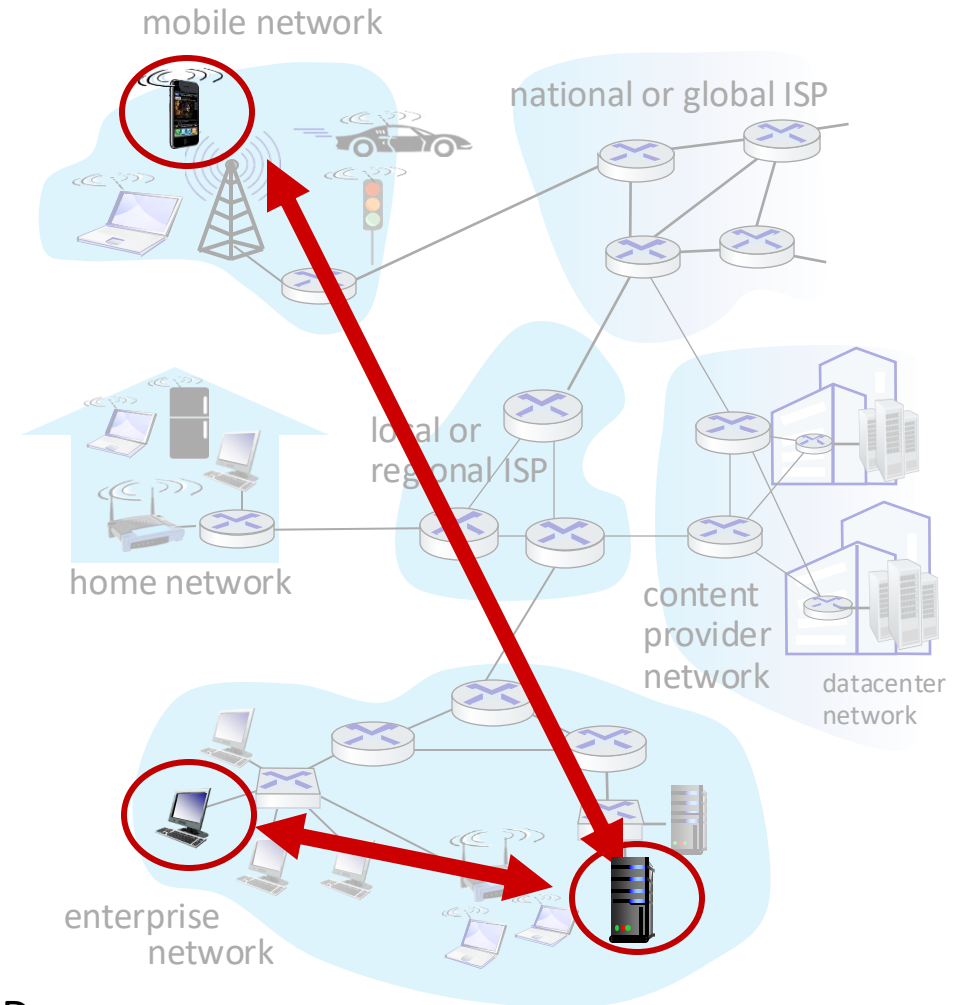
Client-server Paradigm

Server:

- Always-on host
- Permanent IP address
- Often in data centers, for scaling

Clients:

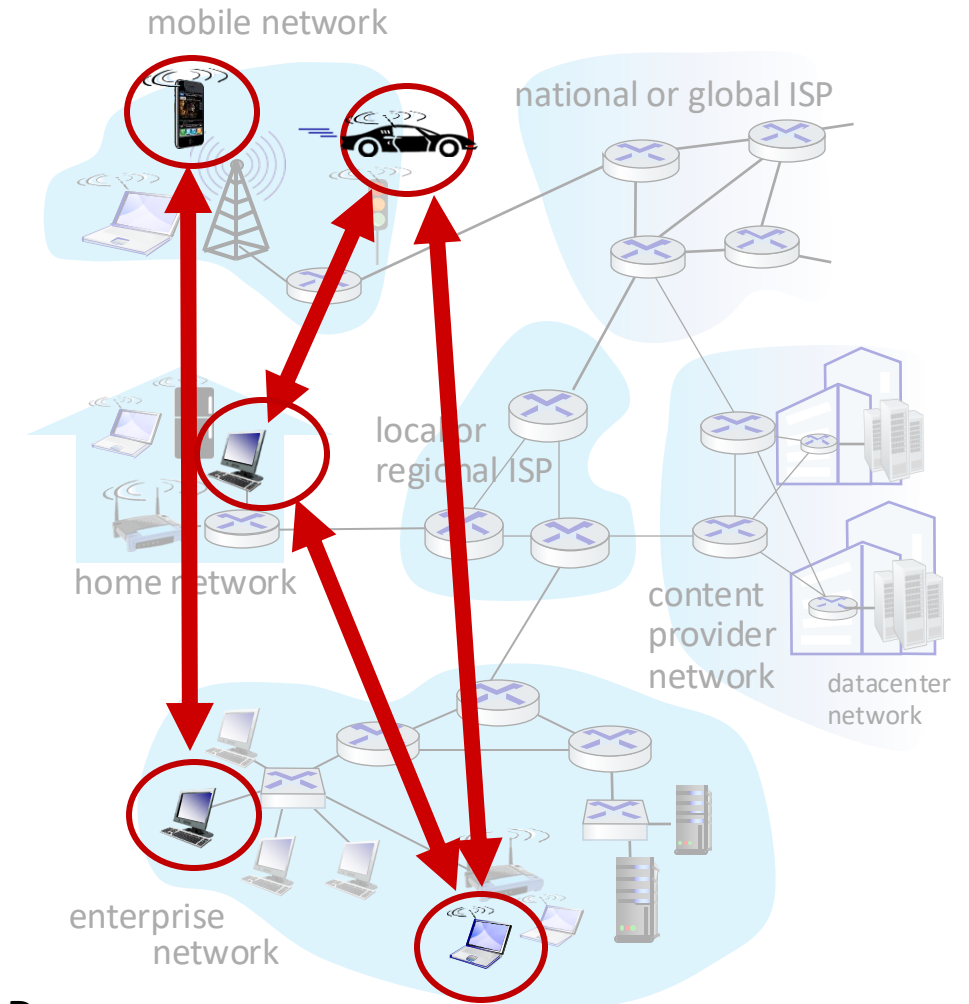
- Contact, communicate with server
- May be intermittently connected
- May have dynamic IP addresses
- Do *not* communicate directly with each other
- Examples: HTTP, IMAP, FTP



Source: Computer Networking: A Top-Down Approach (8th Ed) by Jim Kurose, Keith Ross

Peer-peer Architecture

- No always-on server
- Arbitrary end systems directly communicate
- Peers request service from other peers, provide service in return to other peers
 - *self scalability* – new peers bring new service capacity, as well as new service demands
- Peers are intermittently connected and change IP addresses
 - complex management
- Example: P2P file sharing



Source: Computer Networking: A Top-Down Approach (8th Ed) by Jim Kurose, Keith Ross

Processes Communicating

Process: program running within a host

- Within same host, two processes communicate using **inter-process communication** (defined by OS)
- Processes in different hosts communicate by exchanging **messages**

clients, servers

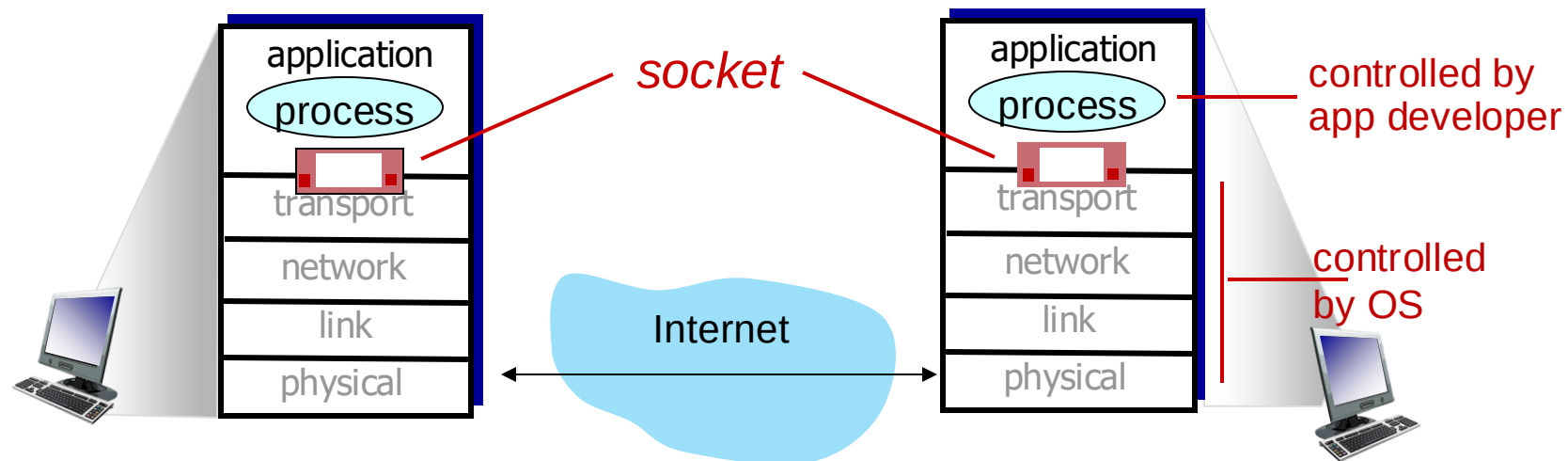
client process: process that initiates communication

server process: process that waits to be contacted

- note: applications with P2P architectures have client processes & server processes

Sockets

- Process sends/receives messages to/from its **socket**
- Socket analogous to door
 - sending process shoves message outdoor
 - sending process relies on transport infrastructure on other side of door to deliver message to socket at receiving process
 - two sockets involved: one on each side



Addressing Processes

- To receive messages, process must have *identifier*
- Host device has unique 32-bit IP address
- Q: does IP address of host on which process runs suffice for identifying the process?
- A: no, *many* processes can be running on same host
- *Identifier* includes both **IP address** and **port numbers** associated with process on host.
- Example port numbers:
 - HTTP server: 80
 - mail server: 25
- To send HTTP message to `www.iitkgp.ac.in` web server:
 - **IP address: 203.110.243.180**
 - **port number: 80**

An Application-layer Protocol Defines

- **Types of messages exchanged,**
 - e.g., request, response
- **Message syntax:**
 - what fields in messages & how fields are delineated
- **Message semantics**
 - meaning of information in fields
- **Rules** for when and how processes send & respond to messages

Open protocols:

- Defined in RFCs, everyone has access to protocol definition
- Allows for interoperability
- e.g., HTTP, SMTP

Proprietary protocols:

- e.g., Skype, Zoom

What Transport Service does an App Need?

Data Integrity

- Some apps (e.g., file transfer, web transactions) require 100% reliable data transfer
- Other apps (e.g., audio) can tolerate some loss

Timing

- Some apps (e.g., Internet telephony, interactive games) require low delay to be “effective”

Throughput

- Some apps (e.g., multimedia) require minimum amount of throughput to be “effective”
- Other apps (“elastic apps”) make use of whatever throughput they get

Security

- Encryption, data integrity, ...

Transport Service Requirements: Common Apps

application	data loss	throughput	time sensitive?
file transfer/download	no loss	elastic	no
e-mail	no loss	elastic	no
Web documents	no loss	elastic	no
real-time audio/video	loss-tolerant	audio: 5Kbps-1Mbps video:10Kbps-5Mbps	yes, 10's msec
streaming audio/video	loss-tolerant	same as above	yes, few secs
interactive games	loss-tolerant	Kbps+	yes, 10's msec
text messaging	no loss	elastic	yes and no

Internet Transport Protocols Services

TCP service:

- *Reliable transport* between sending and receiving process
- *Flow control*: sender won't overwhelm receiver
- *Congestion control*: throttle sender when network overloaded
- *Connection-oriented*: setup required between client and server processes
- *Does not provide*: timing, minimum throughput guarantee, security

UDP service:

- *Unreliable data transfer* between sending and receiving process
- *Does not provide*: reliability, flow control, congestion control, timing, throughput guarantee, security, or connection setup.

Q: why bother? *Why* is there a UDP?

Internet Applications, and Transport Protocols

application	application layer protocol	transport protocol
file transfer/download	FTP [RFC 959]	TCP
e-mail	SMTP [RFC 5321]	TCP
Web documents	HTTP 1.1 [RFC 7320]	TCP
Internet telephony	SIP [RFC 3261], RTP [RFC 3550], or proprietary	TCP or UDP
streaming audio/video	HTTP [RFC 7320], DASH	TCP
interactive games	WOW, FPS (proprietary)	UDP or TCP

Application Layer Interfacing

Application - 1

Application - 2

Application - 3

Application - 4

**End to end
packet delivery**

**Connection
Establishment**

**Reliable Data
Delivery**

**Flow and
Congestion
Control**

**Ordered Packet
Delivery**

UDP

Transport

TCP

Network

Data Link

Application Layer Interfacing



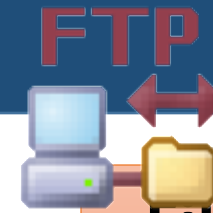
**Name Service
(DNS)**



**Web
(HTTP)**



**Email
(SMTP, POP, IMAP)**



**File Transfer
(FTP)**

**End to end
packet delivery**

UDP

**Connection
Establishment**

**Reliable Data
Delivery**

**Flow and
Congestion
Control**

**Ordered Packet
Delivery**

Transport

TCP

Network

Data Link

The Web – **Hypertext** Transfer Protocol (HTTP)

- **Hypertext** - A way to represent web content (text along with formatting)
- **Hypertext Markup Language (HTML)** - A scripting language to specify web data along with simple formatting (bold, italics, new line).
 - **A way to convert text based information to graphics based information**
- Today's era: Many graphics, scripts and other information are embedded inside HTML – CSS, JavaScript etc.

```
<!DOCTYPE html>
<html>
<!-- created 2010-01-01 -->
<head>
  <title>sample</title>
</head>
<body>
  <p>Voluptatem accusantium
  totam rem aperiam.</p>
</body>
</html>
```

HTML

Differences between HTML Versions

Source: Computer Networks
(5th Edition) by Tanenbaum,
Wetherell

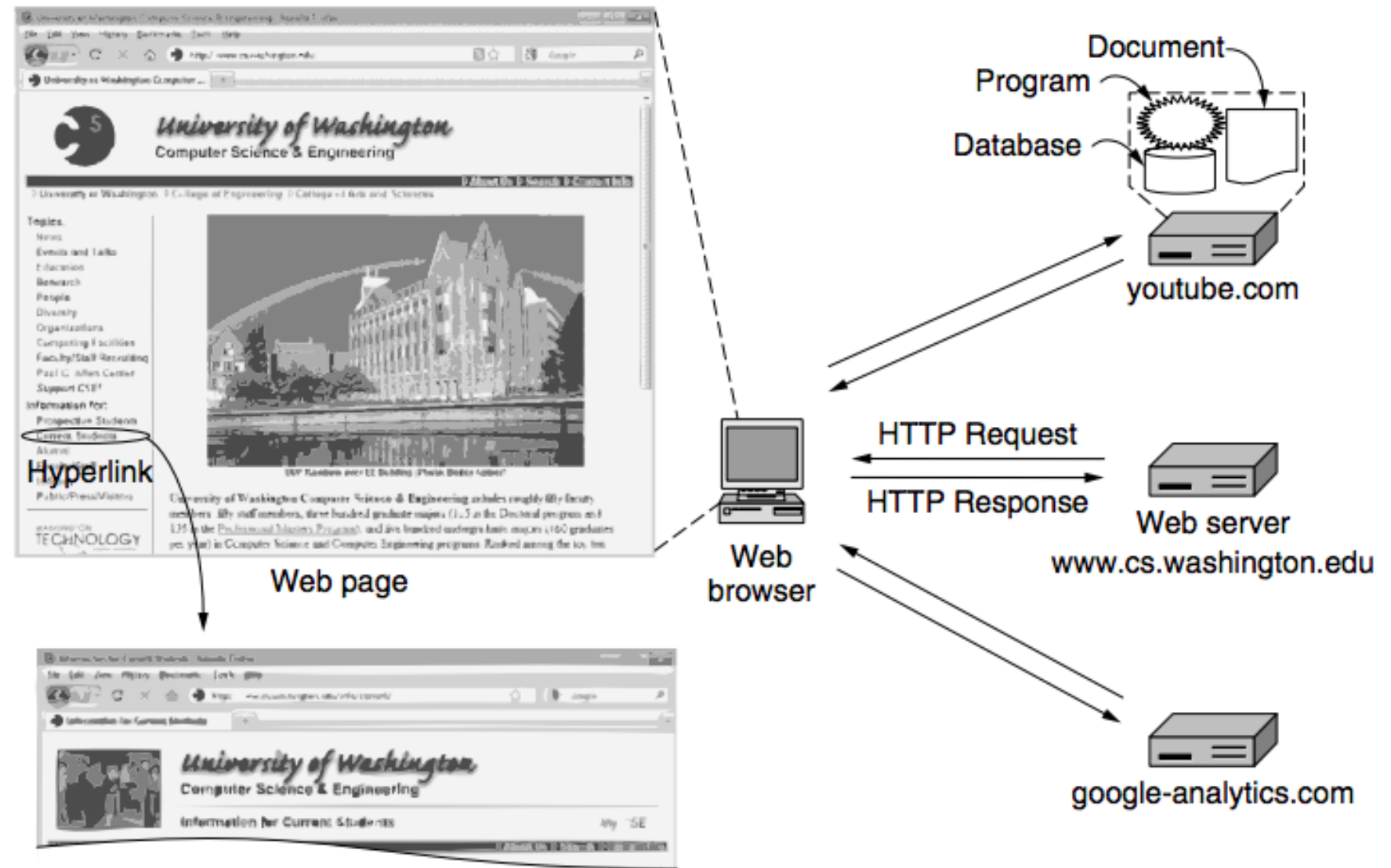
Item	HTML 1.0	HTML 2.0	HTML 3.0	HTML 4.0	HTML 5.0
Hyperlinks	x	x	x	x	x
Images	x	x	x	x	x
Lists	x	x	x	x	x
Active maps & images		x	x	x	x
Forms		x	x	x	x
Equations			x	x	x
Toolbars			x	x	x
Tables			x	x	x
Accessibility features				x	x
Object embedding				x	x
Style sheets				x	x
Scripting				x	x
Video and audio					x
Inline vector graphics					x
XML representation					x
Background threads					x
Browser storage					x
Drawing canvas					x

A History of the Web

- 1989 (CERN – European Center for Nuclear Research) – help large teams to collaborate using a constantly changing collection of reports, blueprints, drawings, photos and other documents
 - The proposal came from Tim Berners-Lee
 - A public demonstration at Hypertext '91 conference
- 1993 – The first graphical browser (Mosaic) developed by Marc Andreessen, University of Illinois
 - Andreessen formed the company Netscape Communications Corp
 - Microsoft developed Internet Explorer – “browser war” between Internet Explorer and Netscape Navigator
- 1994 – CERN and MIT signed an agreement to setup **World Wide Web Consortium (W3C)**

The Web – Architectural Overview

Source: Computer Networks
(5th Edition) by Tanenbaum,
Wetherell



HTTP – The Client Side

- Three questions to be answered for accessing a web page
 - What is the page called? (**courses.html**)
 - Where is the page located? (**cse.iitkgp.ac.in/~sandipc/**)
 - How can the page be accessed? (**http://**)
- **Uniform Resource Locator (URL):** Each page is assigned a URL that effectively serves the page's worldwide name
- **URL** have three components:
 - The protocol
 - The **qualified name** of the machine one which the page is located
 - The path uniquely indicating the specific page

http://cse.iitkgp.ac.in/~sandipc/courses.html

The Steps When You Click <http://cse.iitkgp.ac.in/~sandipc/courses.html>

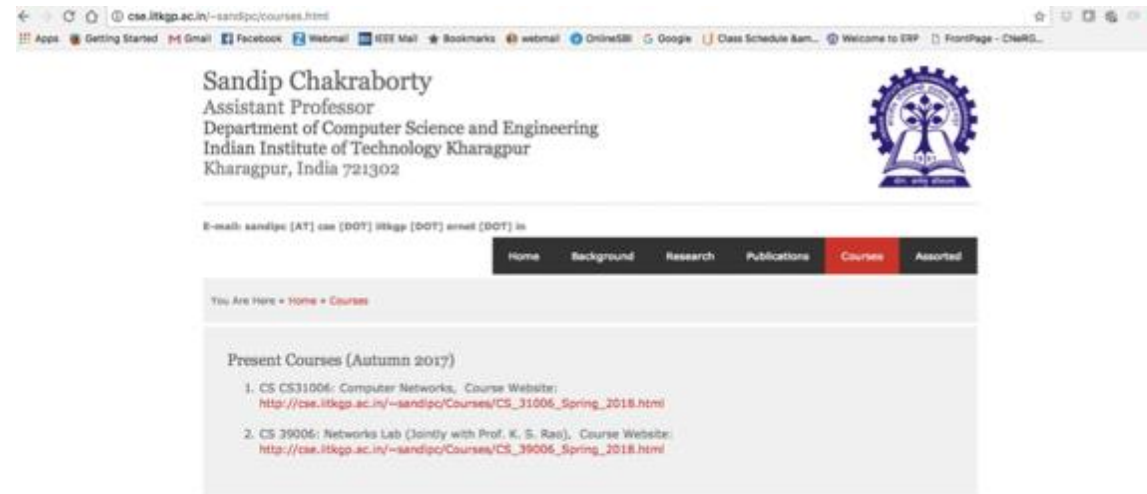
- The browser determines the URL
- The browser asks DNS for the IP address of the server *cse.iitkgp.ac.in*
- DNS replies with 203.110.245.250
- The browser makes a TCP connection to 203.110.245.250 on port 80, the well-known port for the HTTP protocol
- It sends over an HTTP request asking for the page */courses.html*
- The *cse.iitkgp.ac.in* server sends the page as an HTTP response, for example by sending the file */courses.html*
- If the page includes URLs that are needed for display, the browser fetches the other URLs using the same process.

The Steps When You Click <http://cse.iitkgp.ac.in/~sandipc/courses.html>

```
2 <html xmlns="http://www.w3.org/1999/xhtml" xml:lang="EN" lang="EN" dir="ltr">
3 <head profile="http://gmpg.org/xfn/11">
4 <title>Courses | Sandip Chakraborty </title>
5 <meta http-equiv="Content-Type" content="text/html; charset=iso-8859-1" />
6 <meta http-equiv="imagetoolbar" content="no" />
7 <link rel="stylesheet" href="styles/layout.css" type="text/css" />
8 </head>
9 <body id="top">
10 <div class="wrapper">
11 <!-- ##### -->
12 <div id="header">
13 <div class="fl_left">
14 <h1><a href="#">Sandip Chakraborty</a></h1>
15 <h2>Assistant Professor</h2>
16 <h3><a href="http://cse.iitkgp.ernet.in" target="_blank">Department of Computer Science and Engineering</h3>
17 <h3><a href="http://www.iitkgp.ac.in" target="_blank">Indian Institute of Technology Kharagpur</a></h3>
18 <h3>Kharagpur, India 721302</h3>
19 </div>
20 <div class="fl_right"><a href="#"></a></div>
21 <br class="clear" />
22 </div>
23 <!-- ##### -->
24 <div id="topbar">
25 <div class="fl_left">
26 <p><b>E-mail: sandipc [AT] cse [DOT] iitkgp [DOT] ernet [DOT] in</b></p>
27 </div>
28 <div id="topnav">
```

The Steps When You Click <http://cse.iitkgp.ac.in/~sandipc/courses.html>

- The browser displays the page *courses.html*
- The TCP connections are released if there are no other requests to the same server for a short period.



Uniform Resource Identifier (URI)

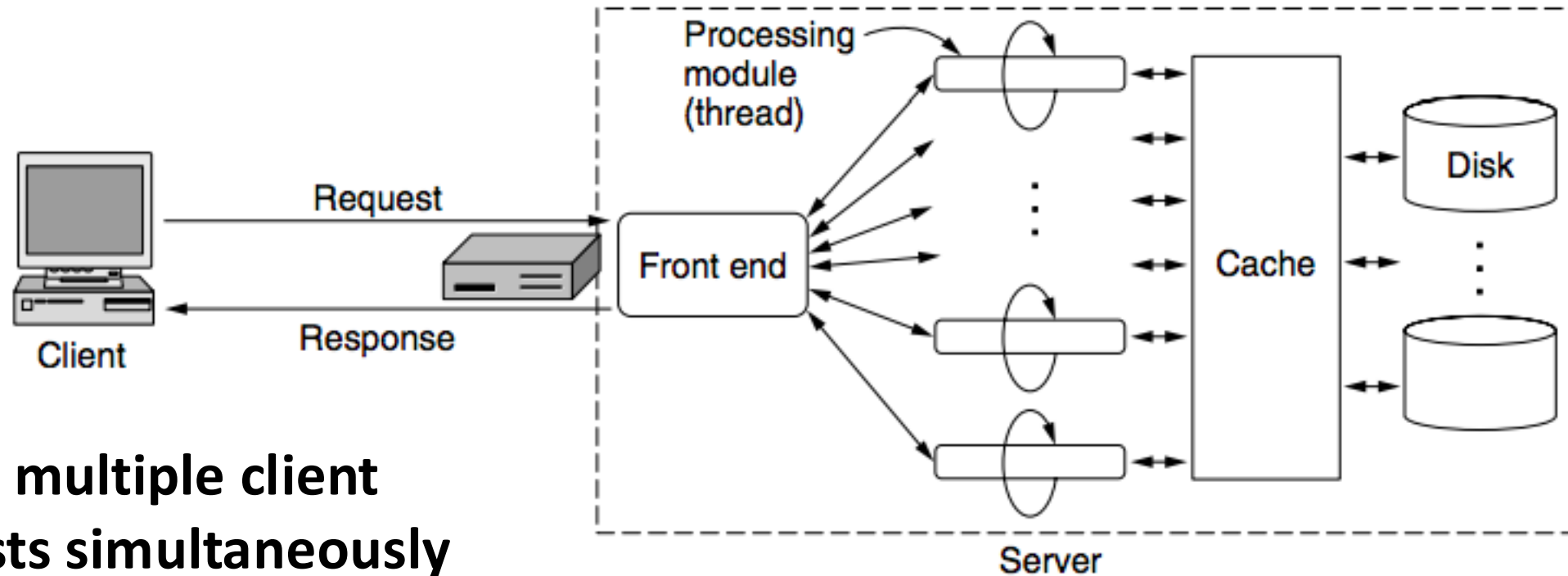
```
2 <html xmlns="http://www.w3.org/1999/xhtml" xml:lang="EN" lang="EN" dir="ltr">
3 <head profile="http://gmpg.org/xfn/11">
4 <title>Courses | Sandip Chakraborty </title>
5 <meta http-equiv="Content-Type" content="text/html; charset=iso-8859-1" />
6 <meta http-equiv="imagetoolbar" content="no" />
7 <link rel="stylesheet" href="styles/layout.css" type="text/css" />
8 </head>
9 <body id="top">
10 <div class="wrapper">
11 <!-- ##### -->
12 <div id="header">
13 <div class="fl_left">
14 <h1><a href="#">Sandip Chakraborty</a></h1>
15 <h2>Assistant Professor</h2>
16 <h3><a href="http://cse.iitkgp.ernet.in" target="_blank">Department of Computer Science and Engineering</h3>
17 <h3><a href="http://www.iitkgp.ac.in" target="_blank">Indian Institute of Technology Kharagpur</a></h3>
18 <h3>Kharagpur, India 751005</h3>
19 </div>
20 <div class="fl_right"><a href="#"></a></div>
21 <br class="clear" />
22 </div>
23 <!-- ##### -->
24 <div id="topbar">
25 <div class="fl_left">
26 <p><b>E-mail: sandipc [AT] cse [DOT] iitkgp [DOT] ernet [DOT] in</b></p>
27 </div>
28 <div id="topnav">
```

- Generalization of the URLs – specifies the pages only or partially refers the pages without complete locations
- `/images/iit_kgp.png` – may become URL `https://cse.iitd.ac.in/images/iit_kgp.png` if accessed from `cse.iitd.ac.in`

HTTP – The Server Side

- Accept a TCP connection from a client (a browser).
- Get the path to the page, which is the name of the file requested.
- Get the file (from disk).
- Sends the content of the file to the client.
- Release (close) the TCP connection.

Multi-Threaded Server



Serves multiple client requests simultaneously

Source: Computer Networks (5th Edition) by Tanenbaum, Wetherell

Connections

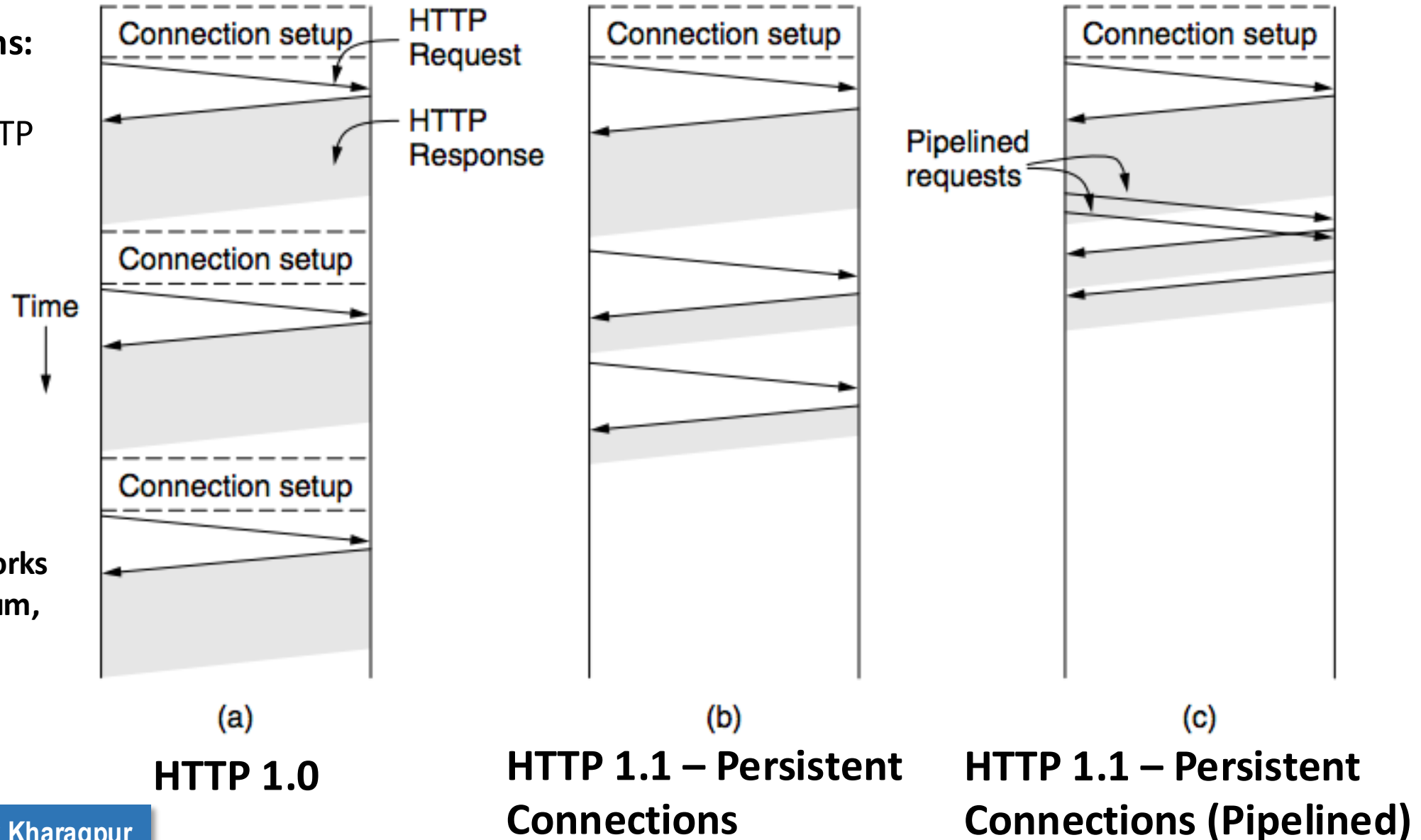
- HTTP uses TCP to set up a connection between the server and the client. In general, HTTP server runs at port 80 (default port) or 8080 (alternate port).
- HTTP 1.0 – After the connections were established, a single request was sent over and a single response was sent back. Then the TCP connections are released.
 - Create separate connections for every content in the web-page. Overhead is high.
- **Persistent Connection** (HTTP 1.1) – send additional requests and additional responses in a single TCP connection (**connection reuse**).
 - It is also possible to **pipeline requests**.

Connections

Persistent Connections:
Set at *keep-alive* information in the HTTP header (HTTP 1.0)

HTTP 1.1 – All connections are by default persistent

Source: Computer Networks (5th Edition) by Tanenbaum, Wetherell

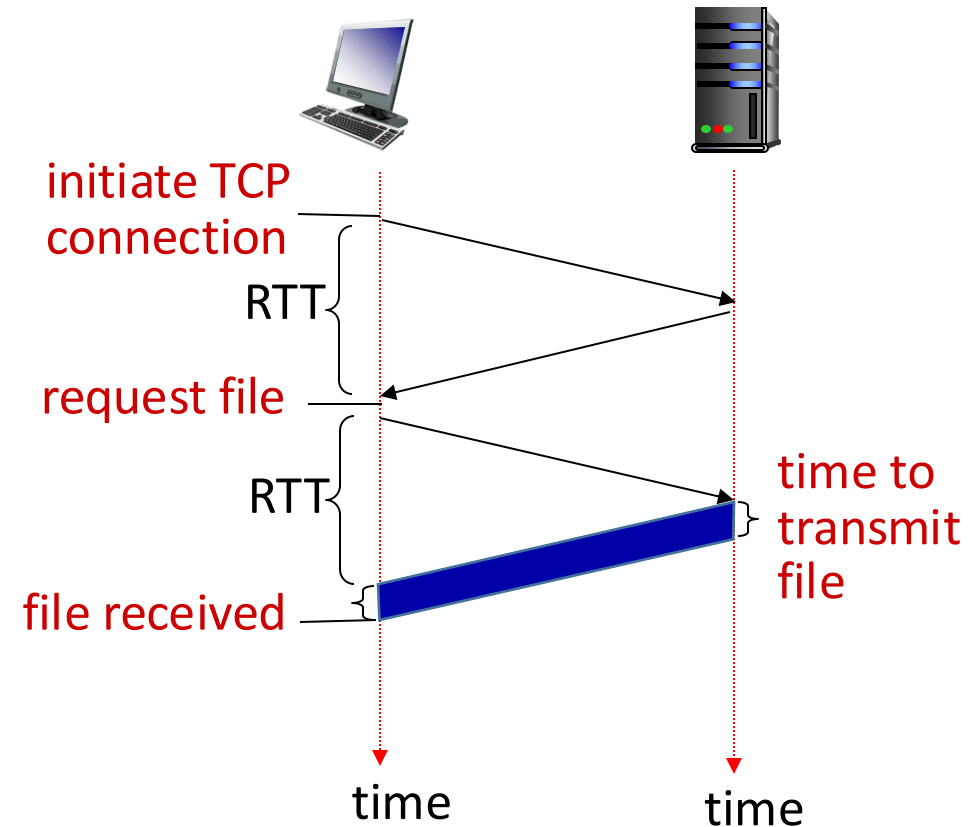


Non-persistent HTTP: Response Time

RTT (definition): time for a small packet to travel from client to server and back

HTTP response time (per object):

- one RTT to initiate TCP connection
- one RTT for HTTP request and first few bytes of HTTP response to return
- object/file transmission time



Non-persistent HTTP response time = $2RTT + \text{file transmission time}$

Persistent HTTP (HTTP 1.1)

Non-persistent HTTP issues:

- Requires 2 RTTs per object
- OS overhead for *each* TCP connection
- Browsers often open multiple parallel TCP connections to fetch referenced objects in parallel

Persistent HTTP (HTTP1.1):

- Server leaves connection open after sending response
- Subsequent HTTP messages between same client/server sent over open connection
- Client sends requests as soon as it encounters a referenced object
- As little as one RTT for all the referenced objects (cutting response time in half)

HTTP Request Methods

- Specifies what a HTTP Request will do
- GET *filename* HTTP/1.1

method	Sp	URL	Sp	Version	Cr	If	Request line Header Line
Header field name			:	value	Cr	If	
:							
Header field name			:	value	Cr	If	
Cr	If						
Entity Body							

Method	Description
GET	Read a Web page
HEAD	Read a Web page's header
POST	Append to a Web page
PUT	Store a Web page
DELETE	Remove the Web page
TRACE	Echo the incoming request
CONNECT	Connect through a proxy
OPTIONS	Query options for a page

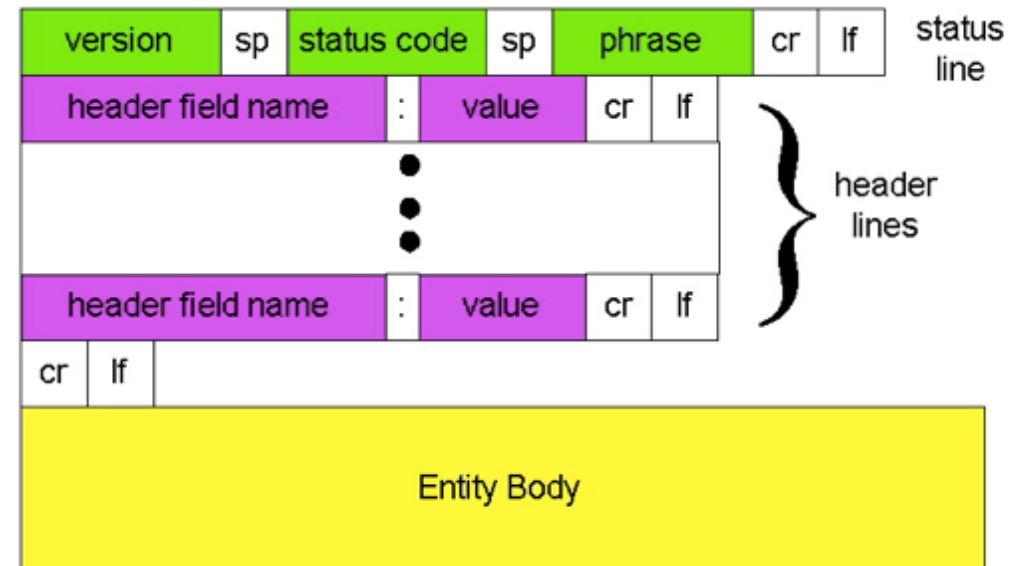
HTTP Request Header Fields (Partial List)

Header	Type	Contents
User-Agent	Request	Information about the browser and its platform
Accept	Request	The type of pages the client can handle
Accept-Charset	Request	The character sets that are acceptable to the client
Accept-Encoding	Request	The page encodings the client can handle
Accept-Language	Request	The natural languages the client can handle
If-Modified-Since	Request	Time and date to check freshness
If-None-Match	Request	Previously sent tags to check freshness
Host	Request	The server's DNS name
Authorization	Request	A list of the client's credentials
Referer	Request	The previous URL from which the request came
Cookie	Request	Previously set cookie sent back to the server

HTTP Response

- Specifies the status of the request message.

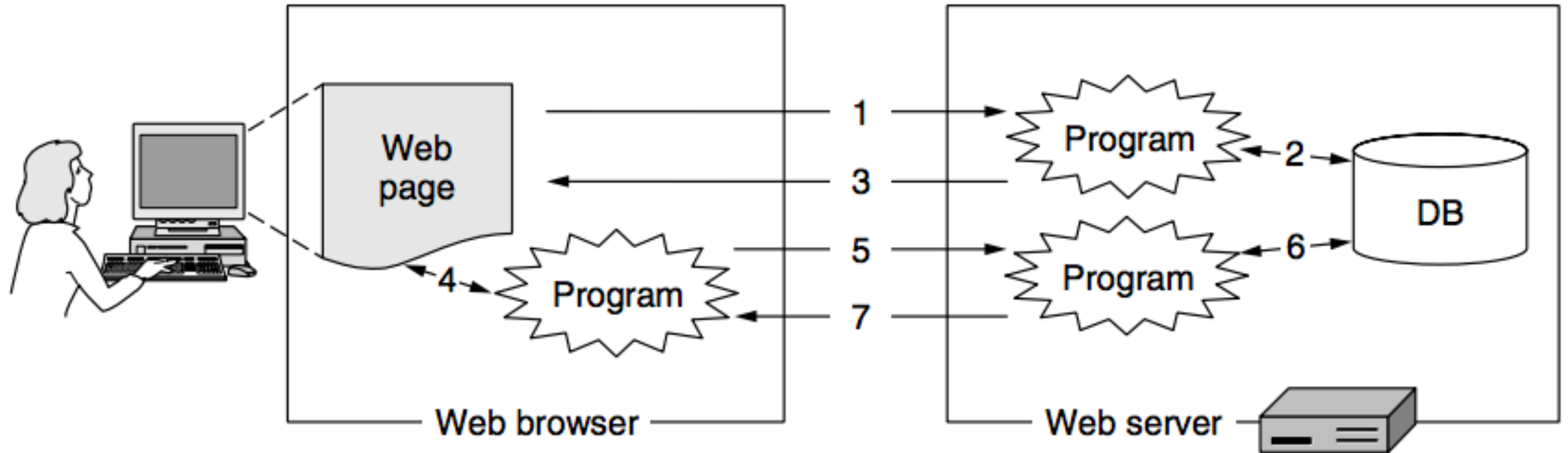
Code	Meaning	Examples
1xx	Information	100 = server agrees to handle client's request
2xx	Success	200 = request succeeded; 204 = no content present
3xx	Redirection	301 = page moved; 304 = cached page still valid
4xx	Client error	403 = forbidden page; 404 = page not found
5xx	Server error	500 = internal server error; 503 = try again later



HTTP Response Header Fields (Partial List)

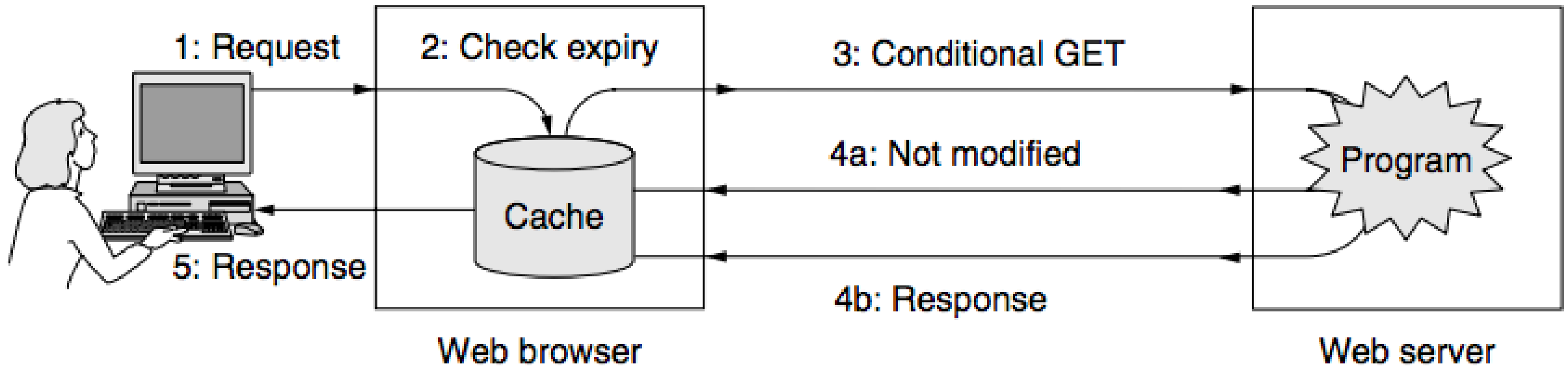
Set-Cookie	Response	Cookie for the client to store
Server	Response	Information about the server
Content-Encoding	Response	How the content is encoded (e.g., <i>gzip</i>)
Content-Language	Response	The natural language used in the page
Content-Length	Response	The page's length in bytes
Content-Type	Response	The page's MIME type
Content-Range	Response	Identifies a portion of the page's content
Last-Modified	Response	Time and date the page was last changed
Expires	Response	Time and date when the page stops being valid
Location	Response	Tells the client where to send its request
Accept-Ranges	Response	Indicates the server will accept byte range requests

Dynamic Web Applications



Source: Computer Networks (5th Edition) by Tanenbaum, Wetherell

HTTP Caching



Source: Computer Networks (5th Edition) by Tanenbaum, Wetherell

Web Caches (aka Proxy Servers)

- Web cache acts as both client and server
 - server for original requesting client
 - client to origin server
- Server tells cache about object's allowable caching in response header:

```
Cache-Control: max-age=<seconds>
```

```
Cache-Control: no-cache
```

Why Web caching?

- Reduce response time for client request
 - Cache is closer to client
- Reduce traffic on an institution's access link
- Internet is dense with caches
 - Enables “poor” content providers to more effectively deliver content

Caching Example

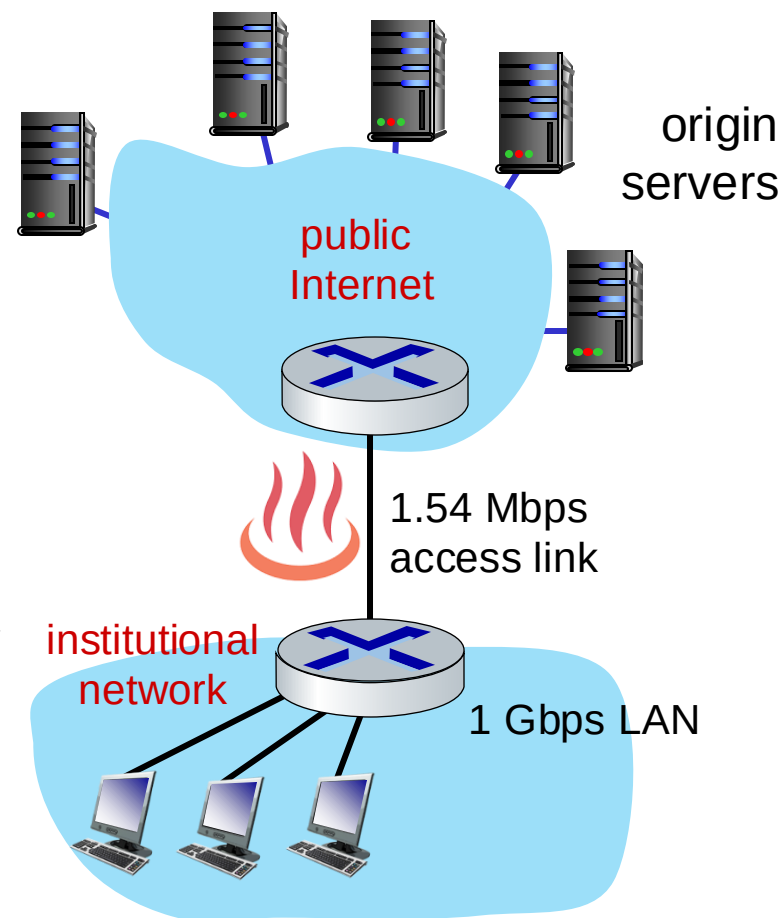
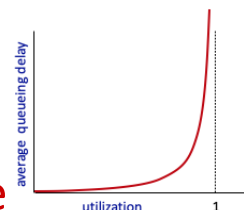
Scenario:

- Access link rate: 1.54 Mbps
- RTT from institutional router to server: 2 sec
- Web object size: 100K bits
- Average request rate from browsers to origin servers: 15/sec
 - avg data rate to browsers: 1.50 Mbps

Performance:

- Access link utilization = .97
- LAN utilization: .0015
- End-end delay = Internet delay + access link delay + LAN delay
= 2 sec + minutes + usecs

problem: large queueing delays at high utilization!



Option 1: Buy a Faster Access Link

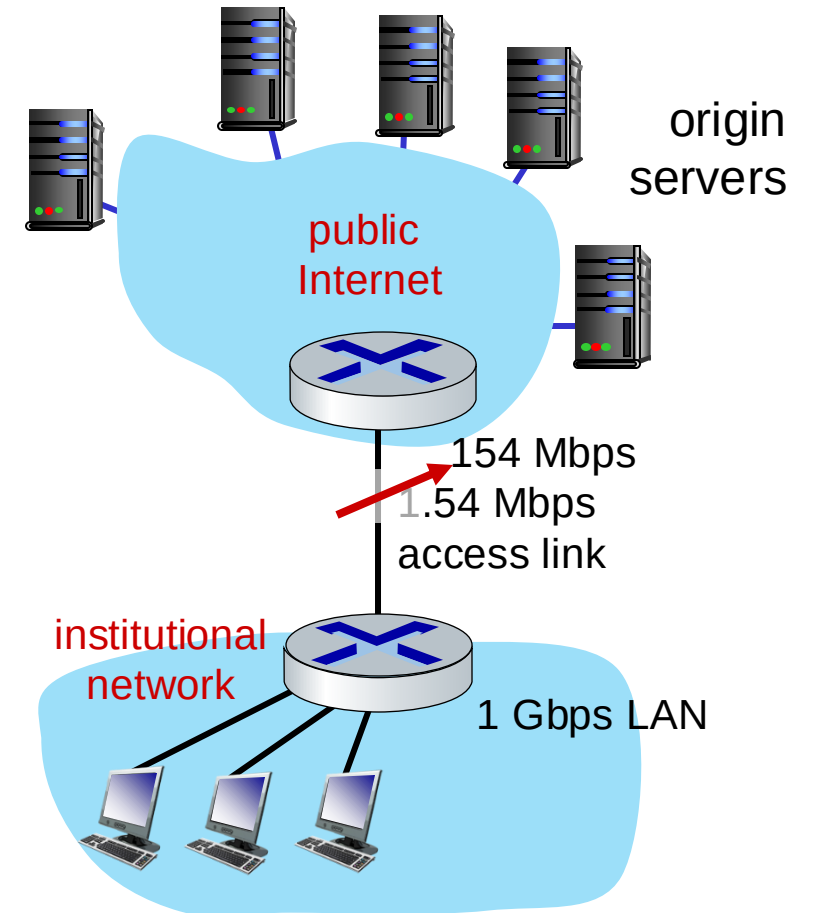
Scenario:

- access link rate: ~~1.54~~ ^{154 Mbps} Mbps
- RTT from institutional router to server: 2 sec
- web object size: 100K bits
- average request rate from browsers to origin servers: 15/sec
 - avg data rate to browsers: 1.50 Mbps

Performance:

- access link utilization = ~~.97~~ ^{.0097}
- LAN utilization: .0015
- end-end delay = Internet delay +
access link delay + LAN delay
= 2 sec + ~~minutes~~ + usecs

Cost: faster access link (expensive!) ^{msecs}



Option 2: Install a Web Cache

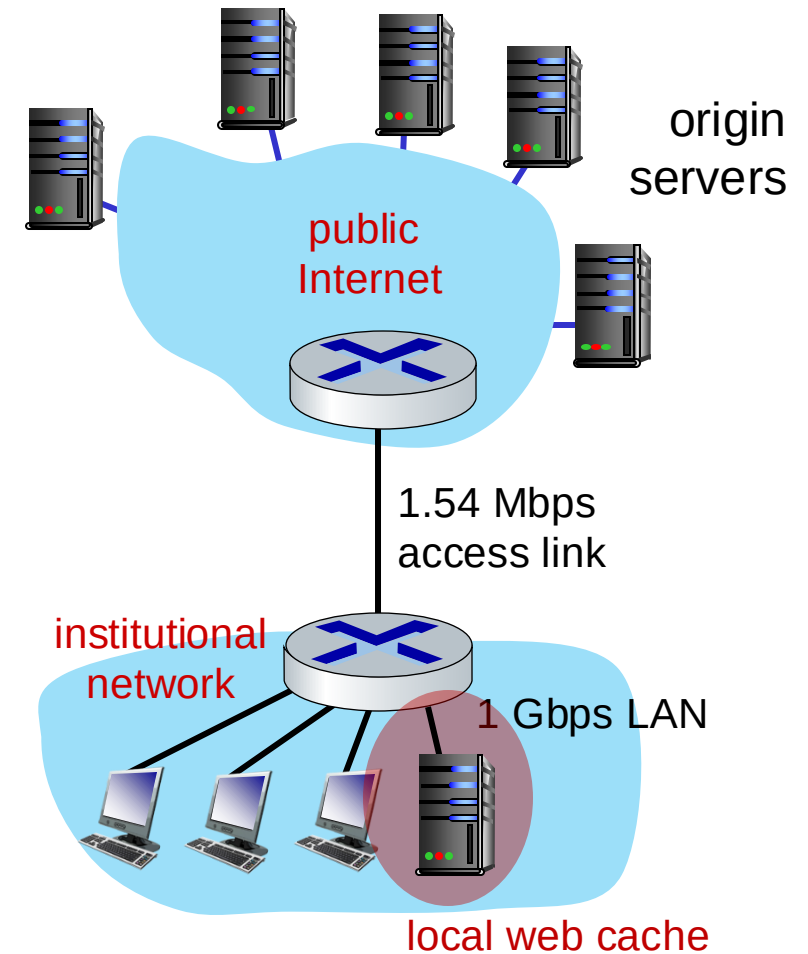
Scenario:

- access link rate: 1.54 Mbps
- RTT from institutional router to server: 2 sec
- web object size: 100K bits
- average request rate from browsers to origin servers: 15/sec
 - avg data rate to browsers: 1.50 Mbps

Cost: web cache (cheap!)

Performance:

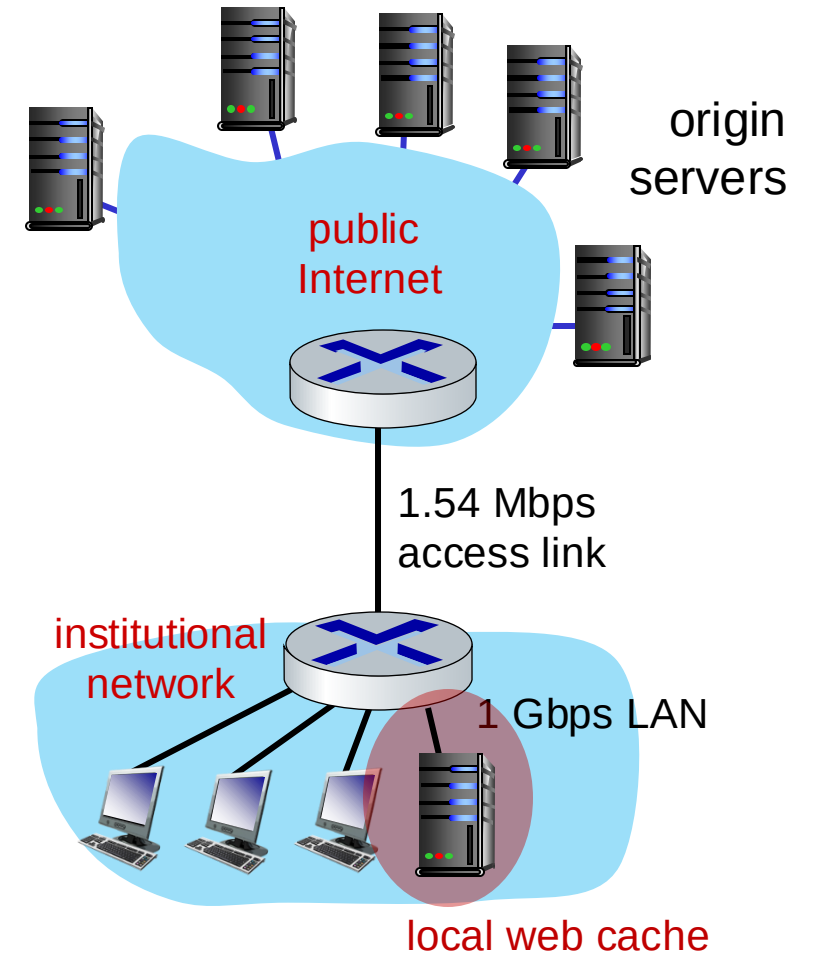
- LAN utilization: .?
 - access link utilization = ?
 - average end-end delay = ?
- How to compute link utilization, delay?*



Calculating Access Link Utilization, End-end Delay with Cache

suppose cache hit rate is 0.4:

- 40% requests served by cache, with low (msec) delay
- 60% requests satisfied at origin
 - rate to browsers over access link
 $= 0.6 * 1.50 \text{ Mbps} = .9 \text{ Mbps}$
 - access link utilization $= 0.9/1.54 = .58$ means low (msec) queueing delay at access link
- average end-end delay:
 $= 0.6 * (\text{delay from origin servers})$
 $+ 0.4 * (\text{delay when satisfied at cache})$
 $= 0.6 (2.01) + 0.4 (\sim \text{msecs}) = \sim 1.2 \text{ secs}$



lower average end-end delay than with 154 Mbps link (and cheaper too!)

Conditional GET

Goal: don't send object if cache has up-to-date cached version

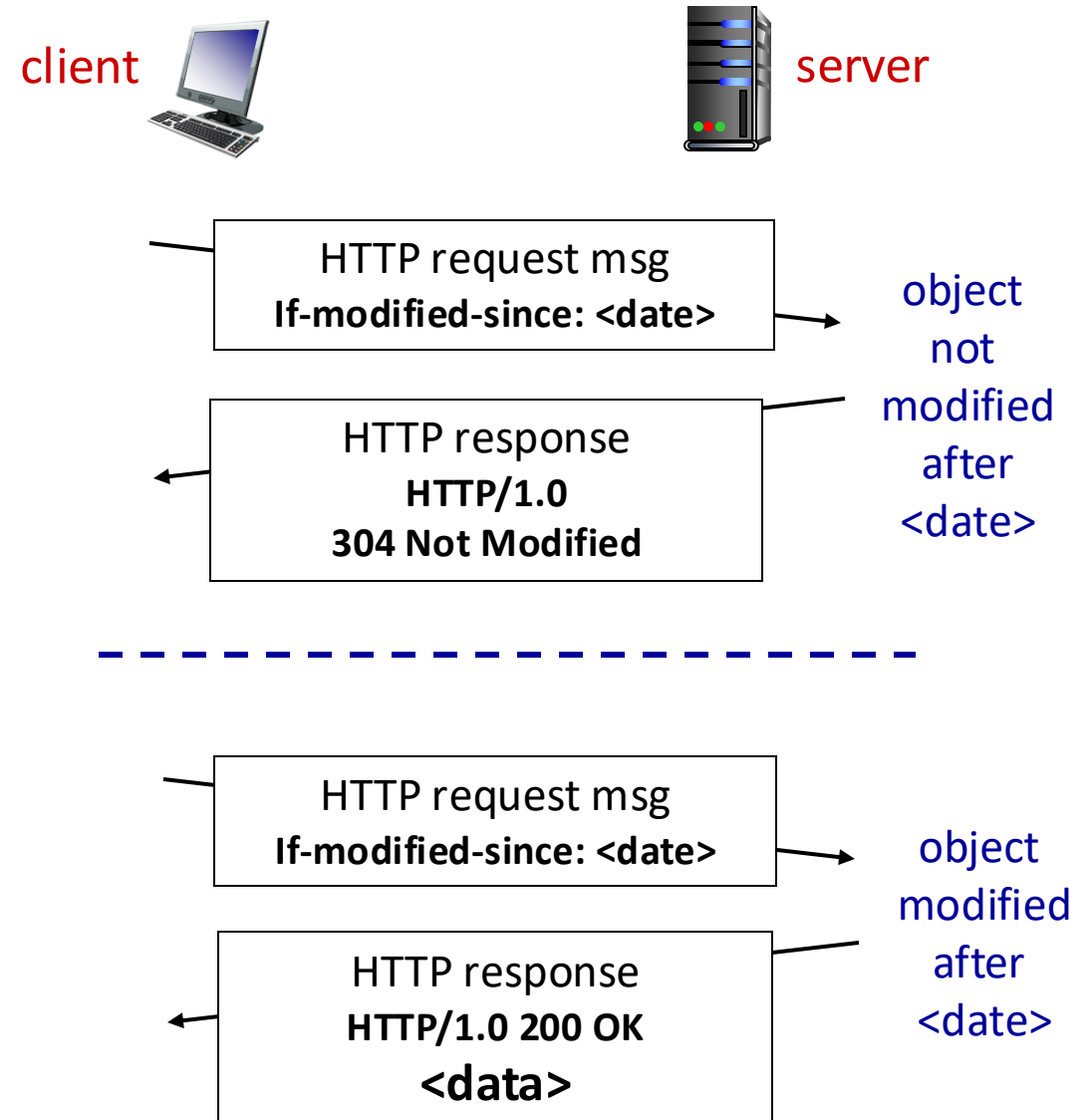
- no object transmission delay (or use of network resources)

■ **Client:** specify date of cached copy in HTTP request

If-modified-since: <date>

■ **Server:** response contains no object if cached copy is up-to-date:

HTTP/1.1 304 Not Modified



Stateless HTTP

HTTP is “stateless”

- server maintains *no* information about past client requests
- every Response corresponds to the previous Request only
- it does not remember any state information, such as last page accessed

- aside*
- protocols that maintain “state” are complex!
- past history (state) must be maintained
 - if server/client crashes, their views of “state” may be inconsistent, must be reconciled

Cookies

- What if we need to be **stateful**?
- Use **Cookies** to store the state information. Client forwards the additional information along with the Request message by reading the cookie.



Maintaining User/Server State: Cookies

Web sites and client browser use *cookies* to maintain some state between transactions

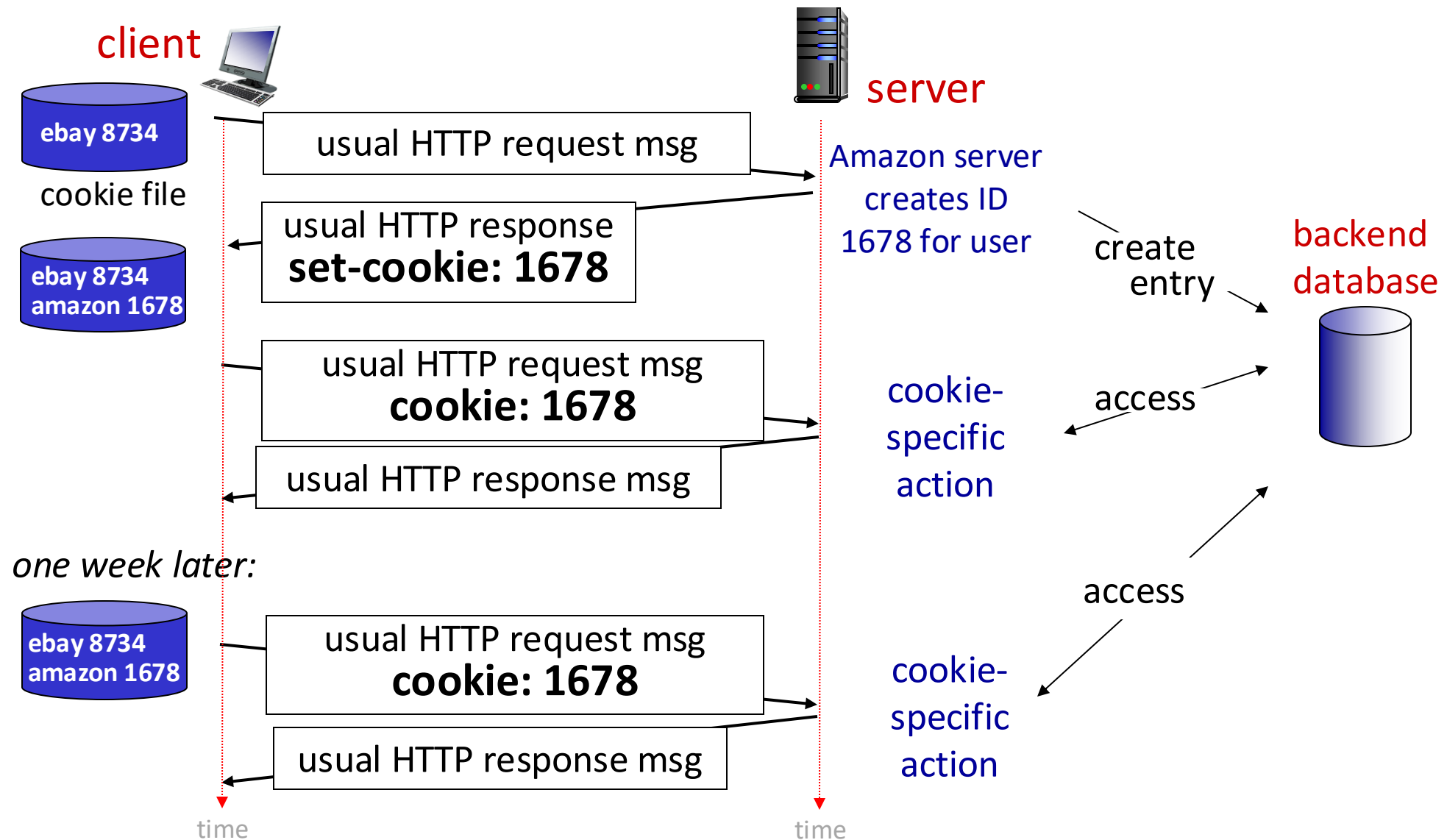
four components:

- 1) cookie header line of HTTP *response* message
- 2) cookie header line in next HTTP *request* message
- 3) cookie file kept on user's host, managed by user's browser
- 4) back-end database at website

Example:

- Susan uses browser on laptop, visits specific e-commerce site for first time
- when initial HTTP requests arrives at site, site creates:
 - unique ID (aka “cookie”)
 - entry in backend DB for ID
- subsequent HTTP requests from Susan to this site will contain cookie ID value, allowing site to “identify” Susan

Maintaining User/Server State: Cookies



HTTP cookies: comments

What cookies can be used for:

- authorization
- shopping carts
- recommendations
- user session state (Web e-mail)

aside

cookies and privacy:

- cookies permit sites to *learn* a lot about you on their site.
- third party persistent cookies (tracking cookies) allow common identity (cookie value) to be tracked across multiple web sites

Application Layer Interfacing



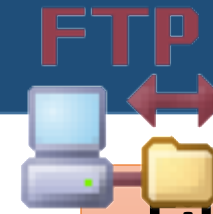
**Name Service
(DNS)**



**Web
(HTTP)**



**Email
(SMTP, POP, IMAP)**



**File Transfer
(FTP)**

**End to end
packet delivery**

UDP

**Connection
Establishment**

**Reliable Data
Delivery**

**Flow and
Congestion
Control**

**Ordered Packet
Delivery**

Transport

TCP

Network

Data Link

DNS: Domain Name System

people: many identifiers:

- Aadhar, name, passport #

Internet hosts, routers:

- IP address (32 bit) - used for addressing datagrams
- “name”, e.g., cs.umass.edu - used by humans

Q: how to map between IP address and name, and vice versa ?

Domain Name System (DNS):

- Assign **Unique names** to an IP address
- ARPANET – a file *hosts.txt* listed all computer names and their IP addresses.
- To resolve hostname conflicts over the Internet, a naming hierarchy needs to be managed
- *distributed database* implemented in hierarchy of many *name servers*
- *application-layer protocol*: hosts, DNS servers communicate to *resolve* names
- *note*: core Internet function, **implemented as application-layer protocol** -> complexity at network’s “edge”

DNS: Services, Structure

DNS services:

- hostname-to-IP-address translation
- host aliasing
 - canonical, alias names
- mail server aliasing
- load distribution
 - replicated Web servers: many IP addresses correspond to one name

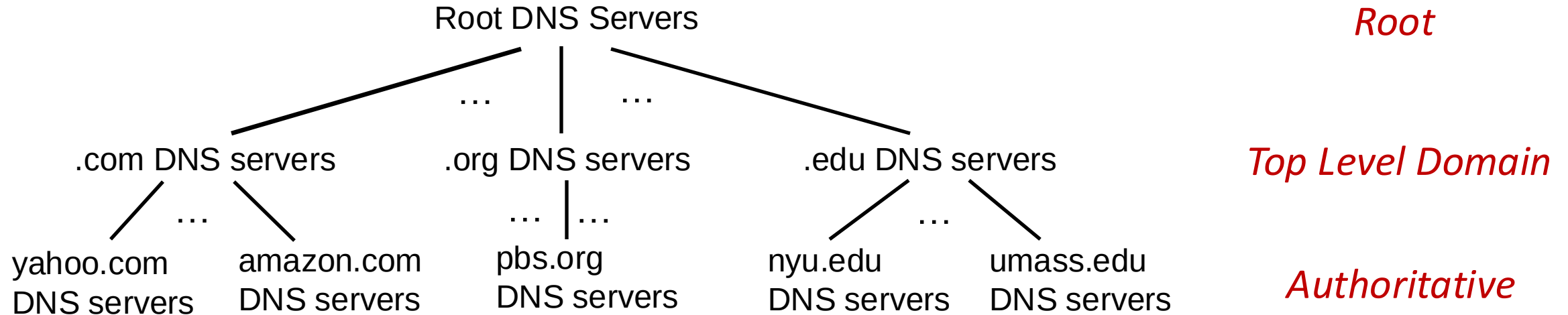
Why not centralize DNS?

- single point of failure
- traffic volume
- distant centralized database
- maintenance

Doesn't scale!

- Telecom DNS servers alone: 600B DNS queries/day
- Akamai DNS servers alone: 2.2T DNS queries/day

DNS: a Distributed, Hierarchical Database

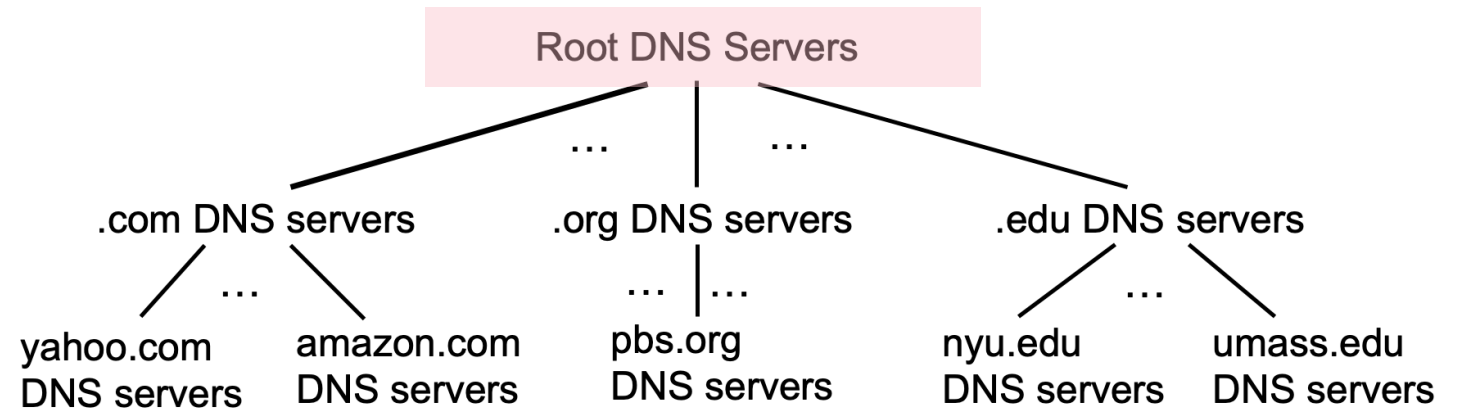


Client wants IP address for www.amazon.com; 1st approximation:

- Client queries root server to find .com DNS server
- Client queries .com DNS server to get amazon.com DNS server
- Client queries amazon.com DNS server to get IP address for www.amazon.com

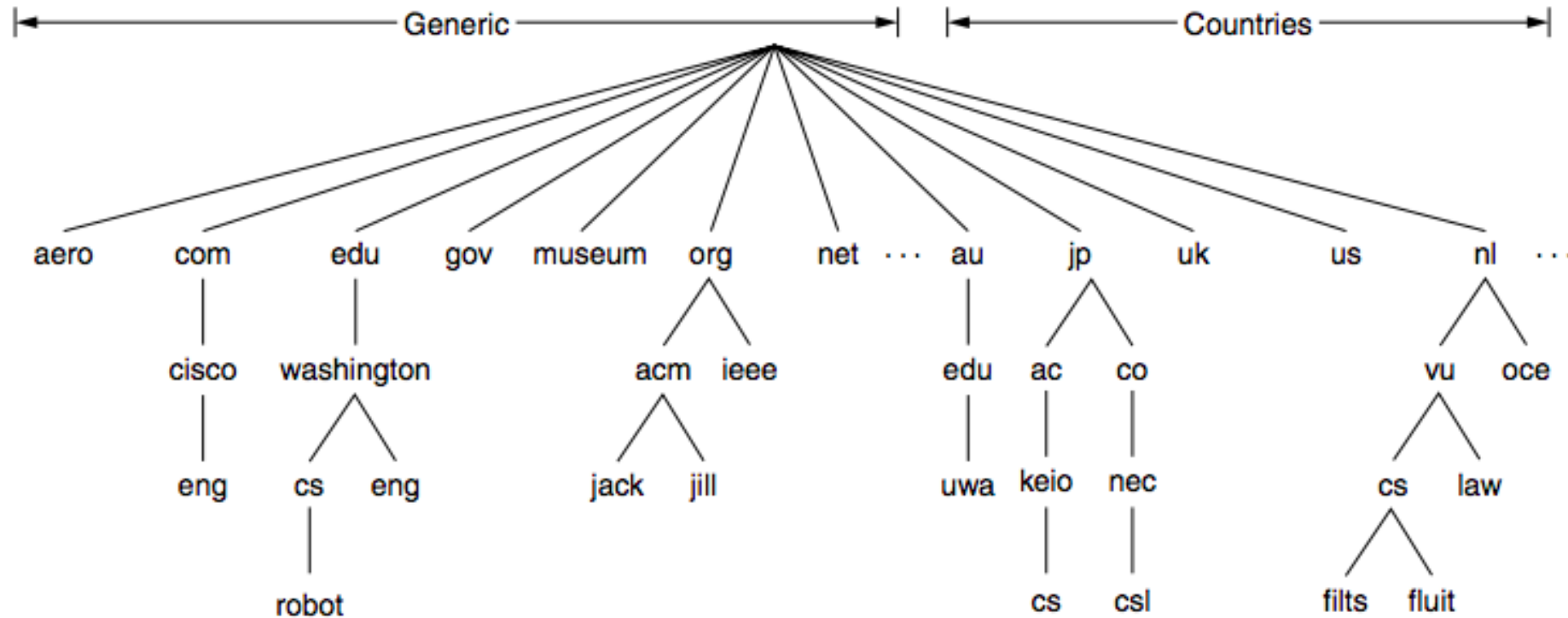
DNS: Root Name Servers

- Official, contact-of-last-resort by name servers that does not directly resolve name
- *Incredibly important* Internet function
 - Internet could not function without it!
- An organization **Internet Corporation for Assigned Names and Numbers (ICANN)** manages root DNS domain



13 logical root name “servers”
worldwide each “server” replicated
many times (~200 servers in US)

Top-Level Domain (TLD) servers



Source: Computer Networks
(5th Edition) by Tanenbaum,
Wetherell

- The top level domains are run by **registrars** appointed by ICANN
- Name registrar for India (.in domain): **registry.in** (National Internet Exchange of India – **NIXI**)

registry.in



[Skip to main content](#)

[Help](#)

[Sitemap](#)

[Login](#)

[Contact Us](#)

[Search](#)

[English | हिन्दी](#)

[A+ | A-](#)



HOME

ABOUT

REGISTRARS

IDN

POLICIES

ANNOUNCEMENTS

Find your .in or .भारत domain right here!

.IN

SEARCH

Domain Name should be a minimum of 3 characters*

Registrars Under registry.in

[Skip to main content](#)[Help](#)[Sitemap](#)[Login](#)[Contact Us](#)[Search](#)[English | हिन्दी](#)[A+ | A-](#)[HOME](#)[ABOUT](#)[REGISTRARS](#)[IDN](#)[POLICIES](#)[ANNOUNCEMENTS](#)

Accredited Registrars

The below registrars are fully accredited, and are authorized to register .IN domain names.

- [List of Registrars with Indian Presence](#)
 - [Top Performers](#)
 - [Other Registrars](#)
- [List of Registrars Located Outside of India](#)
 - [Top Performers](#)
 - [Other Registrars](#)
- [Government registrars](#)
- [IDN accredited registrars](#)

[हिन्दी](#)

Registrars with Indian Presence (Top Performers)

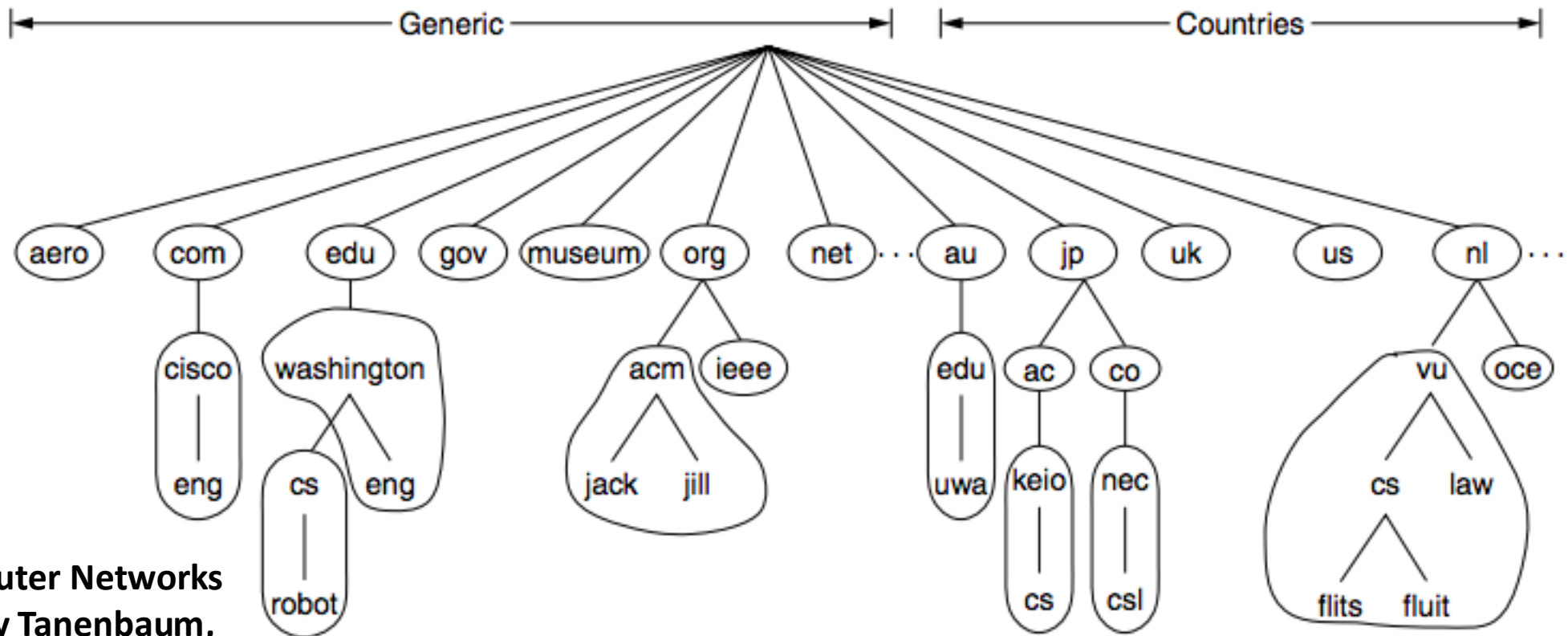
- [Ernet India](#)
- [GoDaddy.com, LLC](#)
- [Business Solutions](#)
- [Silicon House](#)

Elements of DNS (RFC 1034)

- **The Domain Name Space and Resource Records:** Specifications for a tree-structured namespace and data associated with names.
- **Name Servers:** Server programs which hold information about the domain tree's structure and set information
 - A particular name server has complete information about a subset of domain space
 - Name servers know parts of domain tree for which they have complete information
 - A name server is said to be an **AUTHORITY** for this part of the namespace
- **Authoritative DNS servers:**
 - organization's own DNS server(s), providing authoritative hostname to IP mappings for organization's named hosts
 - can be maintained by an organization or a service provider

Name Servers

- Divide registries into **non-overlapping** zones – every zone has a name server



Source: Computer Networks
(5th Edition) by Tanenbaum,
Wetherell

Elements of DNS (RFC 1034)

- **The Domain Name Space and Resource Records:** Specifications for a tree-structured namespace and data associated with names.
- **Name Servers:** Server programs which hold information about the domain tree's structure and set information
 - A particular name server has complete information about a subset of domain space
 - Name servers know parts of domain tree for which they have complete information
- **Resolvers:** Program that extracts information from name servers in response to client requests

Local DNS name servers

- when host makes DNS query, it is sent to its *local* DNS server
 - Local DNS server returns reply, answering:
 - from its local cache of recent name-to-address translation pairs
 - forwarding request into DNS hierarchy for resolution
 - each ISP has local DNS name server; to find yours:
 - MacOS: `% scutil --dns`
 - Windows: `>ipconfig /all`
 - Linux: `nslookup`
- local DNS server doesn't strictly belong to hierarchy

Domain Resource Records

- Every domain has a set of **resource records** associated with it – DNS database

Type	Meaning	Value
SOA	Start of authority	Parameters for this zone
A	IPv4 address of a host	32-Bit integer
AAAA	IPv6 address of a host	128-Bit integer
MX	Mail exchange	Priority, domain willing to accept email
NS	Name server	Name of a server for this domain
CNAME	Canonical name	Domain name
PTR	Pointer	Alias for an IP address
SPF	Sender policy framework	Text encoding of mail sending policy
SRV	Service	Host that provides it
TXT	Text	Descriptive ASCII text

Domain Resource Records

- **Domain_name:** Domain to which the record applies.
- **Time_to_live:** Time for which this record is active – volatile records may be assigned a small value
- **Class:** Normally IN – Internet resources
- **Type:** What type of record it is
- **Value:** Value of the record (IP address for A record type)

Type	Meaning	Value
SOA	Start of authority	Parameters for this zone
A	IPv4 address of a host	32-Bit integer
AAAA	IPv6 address of a host	128-Bit integer
MX	Mail exchange	Priority, domain willing to accept email

Domain Resource Records

- **A Type Records:**

cse.iitkgp.ac.in 86400 IN A 203.110.245.250

- **CNAME Type Records:**

iitkgp.ac.in 86400 IN CNAME www.iitkgp.ac.in

- **PTR Type Records:**

www.iitkgp.ernet.in 86400 IN PTR www.iitkgp.ac.in

Sample DNS Database

Source: Computer Networks
(5th Edition) by Tanenbaum,
Wetherell

; Authoritative data for cs.vu.nl

cs.vu.nl.	86400	IN	SOA	star boss (9527,7200,7200,241920,86400)
cs.vu.nl.	86400	IN	MX	1 zephyr
cs.vu.nl.	86400	IN	MX	2 top
cs.vu.nl.	86400	IN	NS	star

star	86400	IN	A	130.37.56.205
zephyr	86400	IN	A	130.37.20.10
top	86400	IN	A	130.37.20.11
www	86400	IN	CNAME	star.cs.vu.nl
ftp	86400	IN	CNAME	zephyr.cs.vu.nl
flits	86400	IN	A	130.37.16.112
flits	86400	IN	A	192.31.231.165
flits	86400	IN	MX	1 flits
flits	86400	IN	MX	2 zephyr
flits	86400	IN	MX	3 top

rowboat		IN	A	130.37.56.201
		IN	MX	1 rowboat
		IN	MX	2 zephyr

little-sister		IN	A	130.37.62.23
---------------	--	----	---	--------------

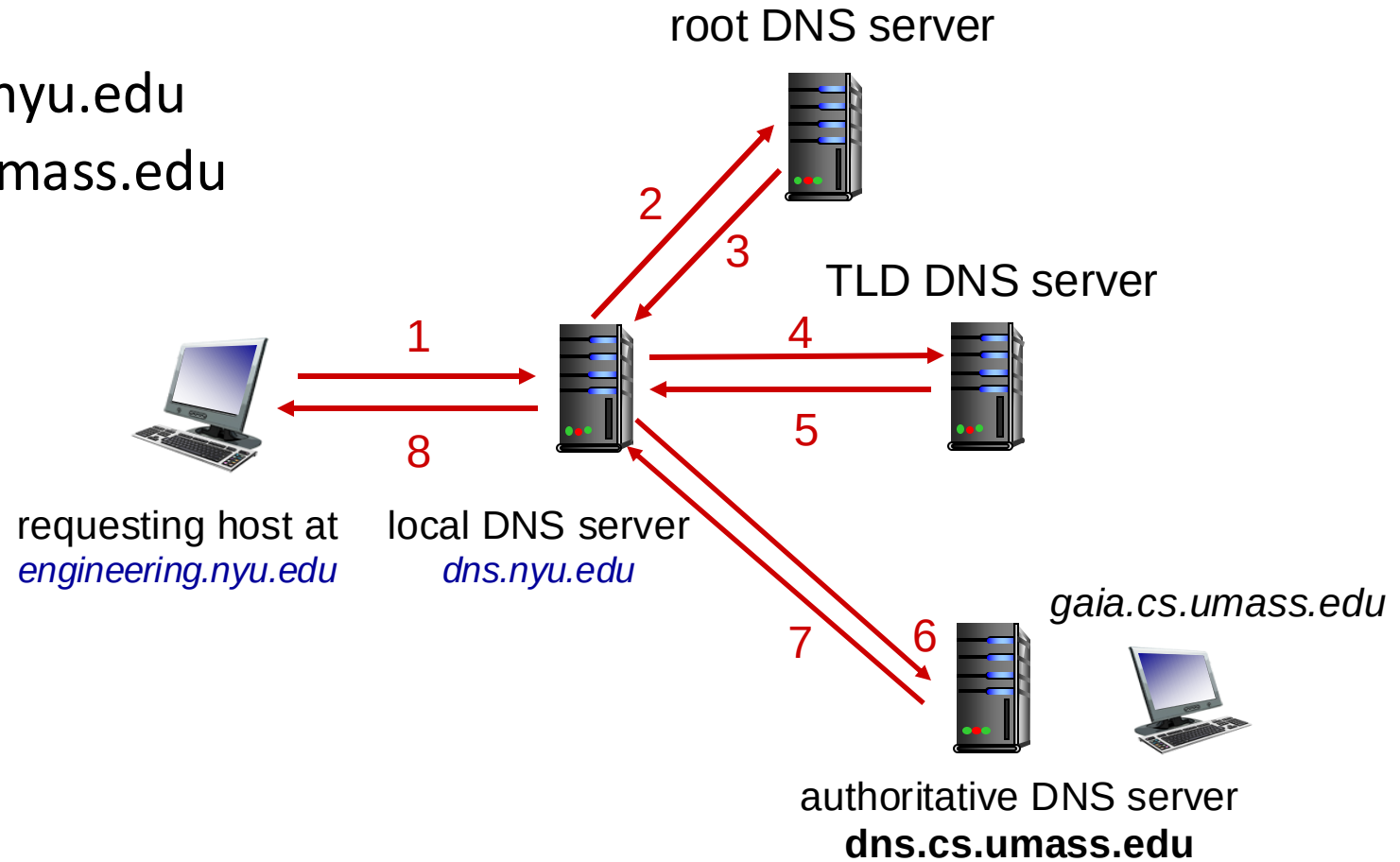
laserjet		IN	A	192.31.231.216
----------	--	----	---	----------------

DNS name resolution: iterated query

Example: host at `engineering.nyu.edu` wants IP address for `gaia.cs.umass.edu`

Iterated query:

- contacted server replies with name of server to contact
- “I don’t know this name, but ask this server”

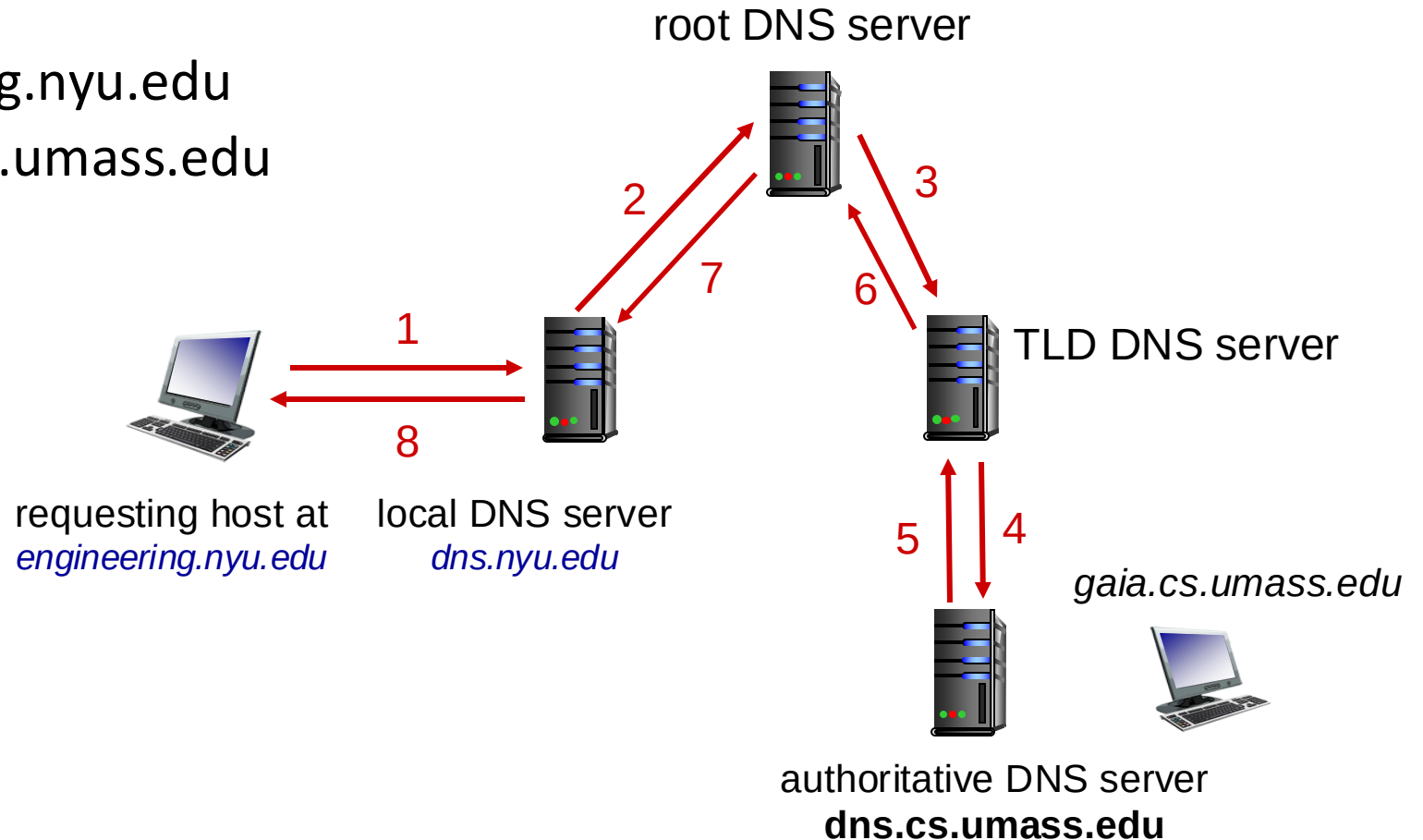


DNS name resolution: recursive query

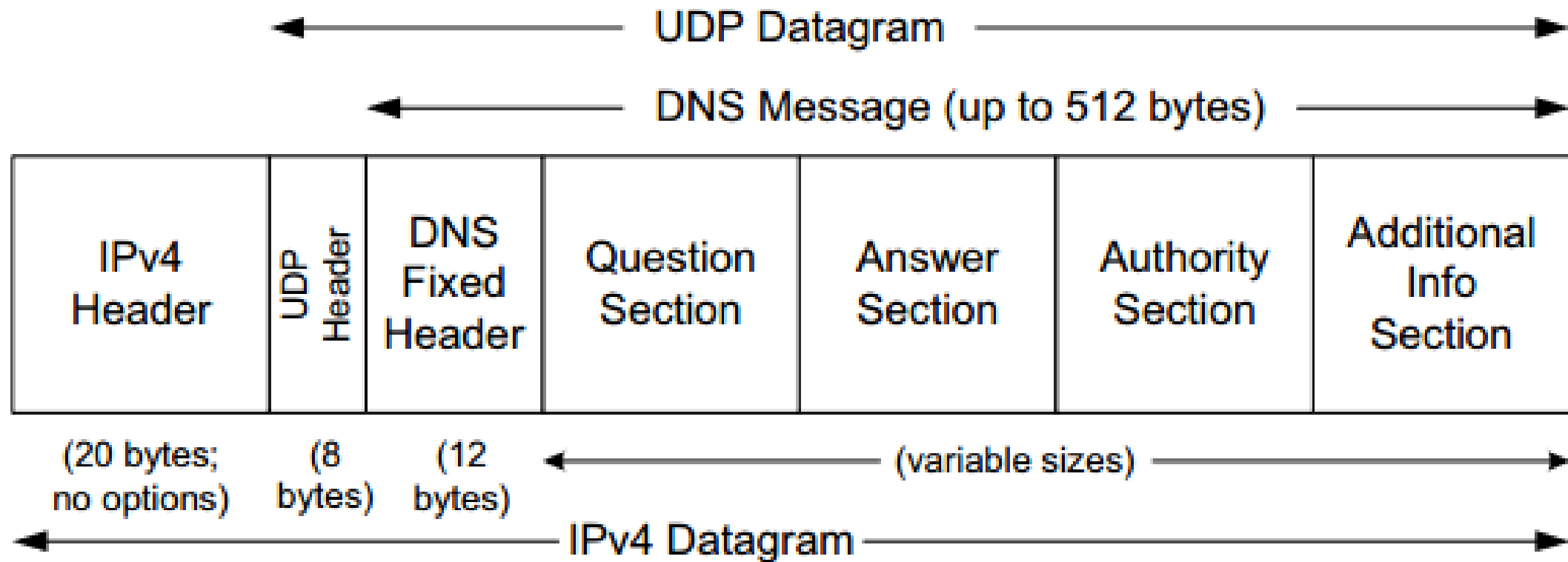
Example: host at `engineering.nyu.edu` wants IP address for `gaia.cs.umass.edu`

Recursive query:

- puts burden of name resolution on contacted name server
- heavy load at upper levels of hierarchy?



DNS Packet Structure



Name Resolution (nslookup)

```
Sandips-Air:MoPreLiCo MajorRevision Sandip$ nslookup www.washington.edu
```

```
Server:          172.16.1.164
```

```
Address:         172.16.1.164#53
```

One of the name servers for IITKGP

```
Non-authoritative answer:
```

```
Name:   www.washington.edu
```

```
Address: 128.95.155.134
```

```
Name:   www.washington.edu
```

```
Address: 128.95.155.135
```

```
Name:   www.washington.edu
```

```
Address: 128.95.155.198
```

Name Resolution (dig)

An **authoritative record** is one that comes from the authority that manages the record, and thus is always correct

UDP Message for 4096 bytes

```
Sandips-Air:MoPreLiCo_MajorRevision Sandip$ dig www.tum.de
```

```
; <<>> DiG 9.9.7-P3 <<>> www.tum.de
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 29257
;; flags: qr rd ra ad; QUERY: 1, ANSWER: 2, AUTHORITY: 3, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4096
;; QUESTION SECTION:
;www.tum.de.                IN      A

;; ANSWER SECTION:
www.tum.de.                 2822    IN      CNAME   wwwv1.tum.de.
wwwv1.tum.de.              2822    IN      A       129.187.255.228

;; AUTHORITY SECTION:
tum.de.                     2822    IN      NS       dns3.lrz.eu.
tum.de.                     2822    IN      NS       dns2.lrz.bayern.
tum.de.                     2822    IN      NS       dns1.lrz.de.

;; Query time: 54 msec
;; SERVER: 172.16.1.164#53(172.16.1.164)
;; WHEN: Mon Jan 15 00:36:01 IST 2018
;; MSG SIZE rcvd: 152
```

Why DNS Uses UDP

- UDP is much faster. TCP requires handshake time. DNS uses a cascading approach for name resolution. With TCP, for every message, a connection setup is required.
- DNS requests and responses are generally very small and fits well within one UDP segment.
- **UDP is not reliable.** In DNS, reliability is ensured at the application layer. After timeout, the DNS client sends back the requests. After few consecutive timeouts (can be set at the client), the request is aborted with an error.

Getting your info into the DNS

Example: you have created a new startup “KGP Ideas”

- register name kgpideas.com at *DNS registrar* (e.g., GoDaddy)
 - provide names, IP addresses of authoritative name server (primary and secondary)
 - registrar inserts NS, A RRs into .com TLD server:
(kgpideas.com, dns1.kgpideas.com, NS)
(dns1.kgpideas.com, 212.212.212.1, A)
- create authoritative server locally with IP address 212.212.212.1
 - type A record for www.kgpideas.com
 - type MX record for kgpideas.com

Application Layer Interfacing



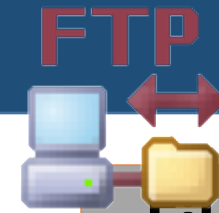
**Name Service
(DNS)**



**Web
(HTTP)**



**Email
(SMTP, POP, IMAP)**



**File Transfer
(FTP)**

**End to end
packet delivery**

UDP

**Connection
Establishment**

**Reliable Data
Delivery**

**Flow and
Congestion
Control**

**Ordered Packet
Delivery**

Transport

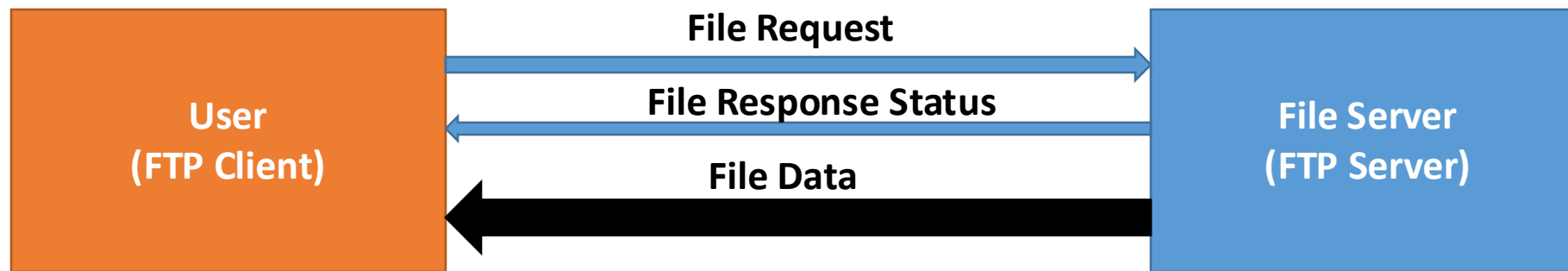
TCP

Network

Data Link

File Transfer Protocol (FTP)

- Is built on a client-server model (RFC 959)
 - The client requests for the file or send the file to the server
 - The server responses with the file data or store the file at the file server



- Works in two modes – **Active and Passive**

Active and Passive Modes of File Transfer

FTP server uses two different ports:

Port 21 (Command or Control Port): For command message transfer

Port 20 or Client assigned (Data Port): For data transfer

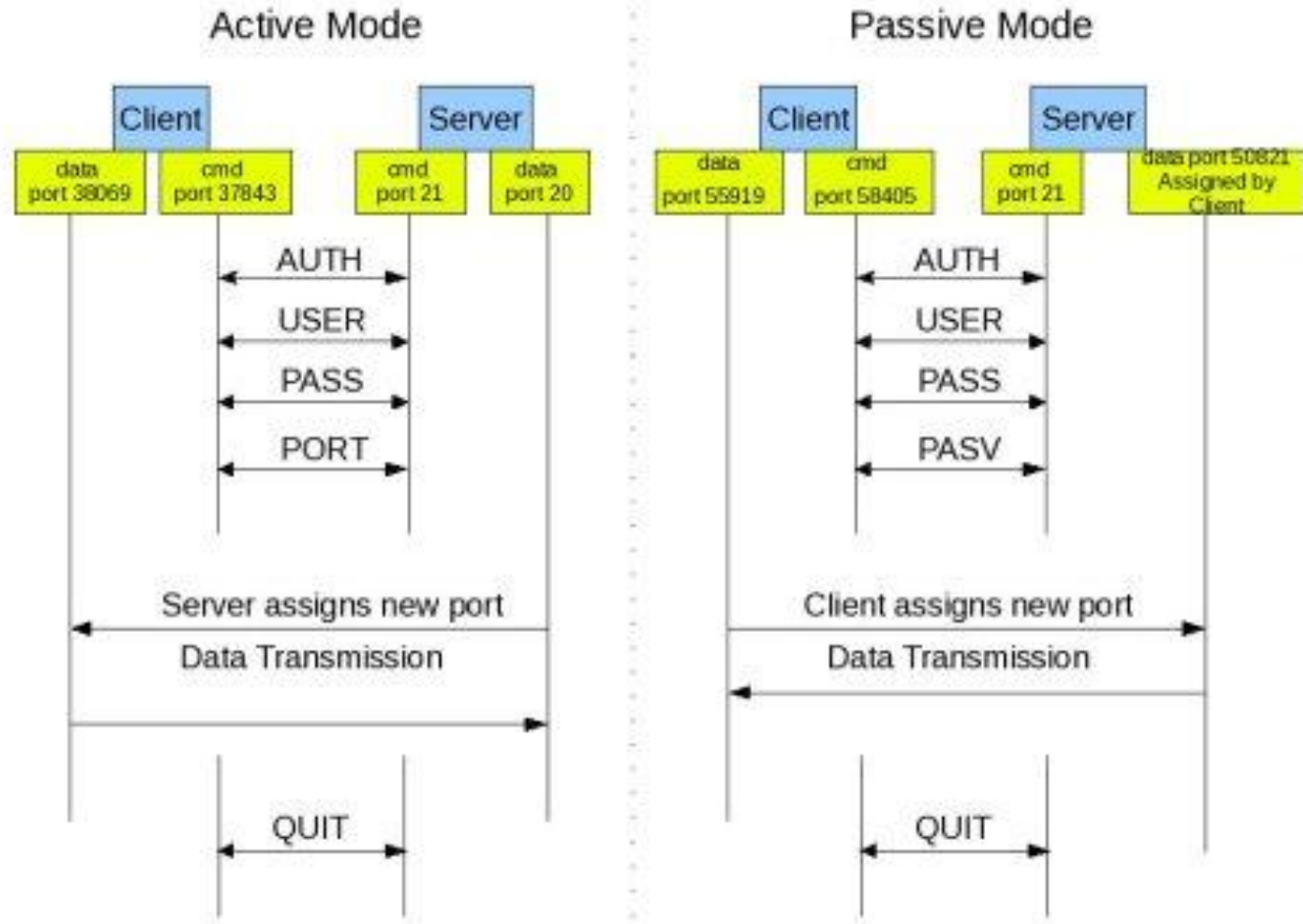


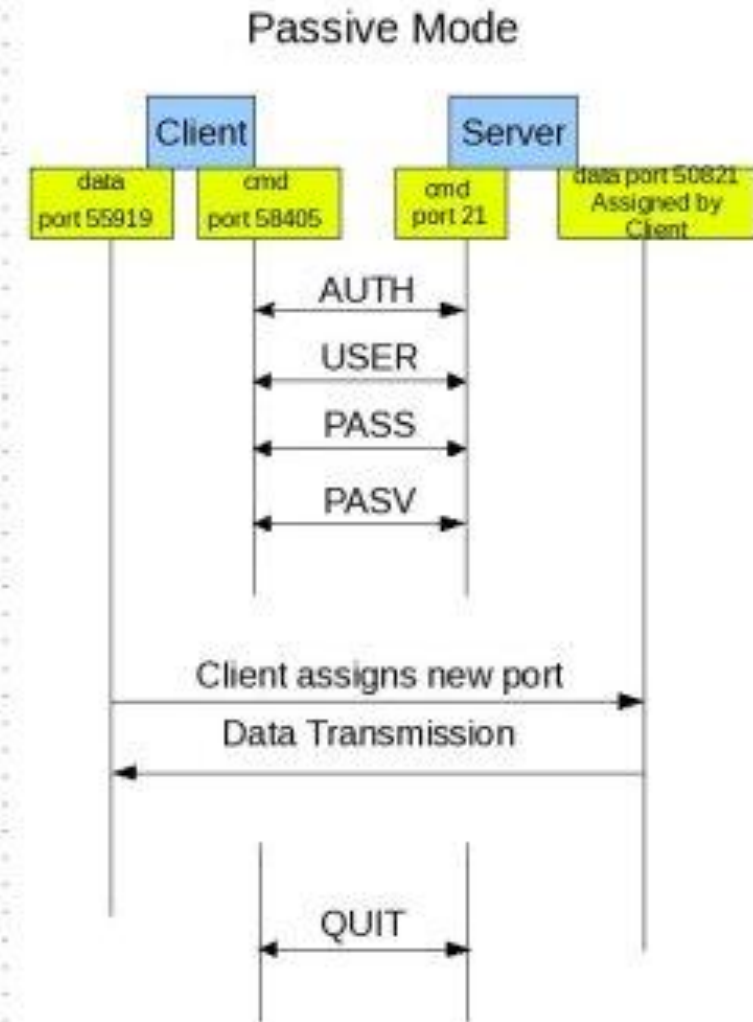
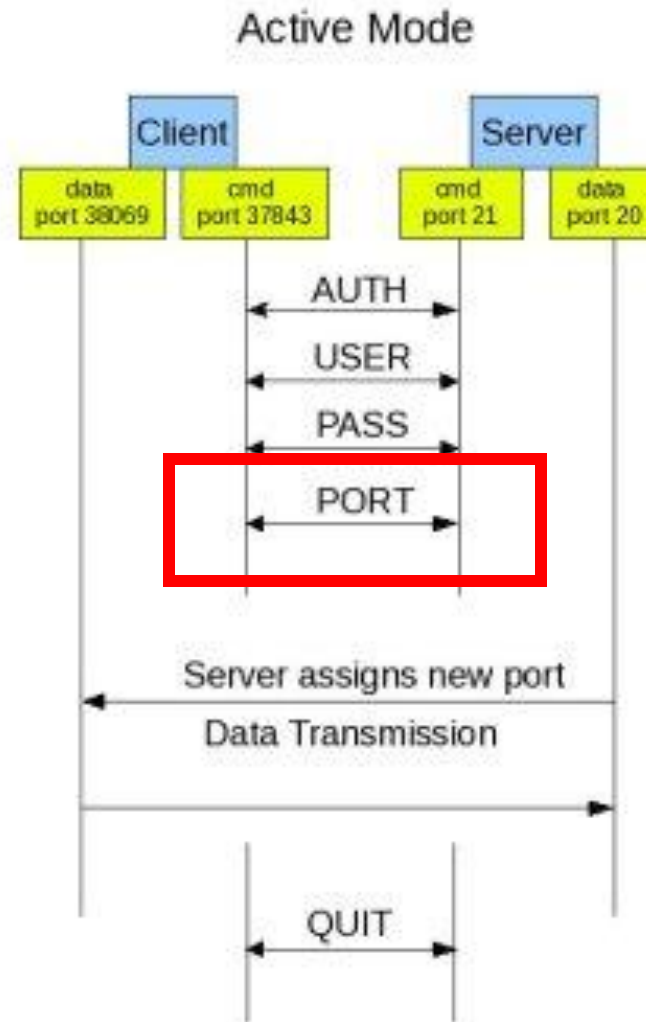
Image Source: <http://henrydu.com/blog/how-to/ftp-active-mode-vs-passive-mode-106.html>

Why There are Two Channels – Command Channel and Data Channel

- Specifically to avoid busy waiting, and keep the command channel lightweight.
- **You can always use a multiplexing between command/control and data, but** FTP is used for large file transfer; if command channel is used for data transfer as well, the commands for other clients may experience a higher queuing delay while one client is being served.
- The clients can continue sending and receiving control information while data transfer is being take place

Why There are Two Modes in FTP?

Active Mode: Client informs the port number where it is listening, and the server initiates the TCP connection to that port (TCP server is running at the client side)



What If the client is behind a firewall and can not accept a connection?

Why There are Two Modes in FTP?

Passive Mode: The server selects a random port, and the client initiates a TCP connection to that server port.

The server can serve multiple clients at different server data ports through different threads.

The clients always initiate the command and the data transfer.

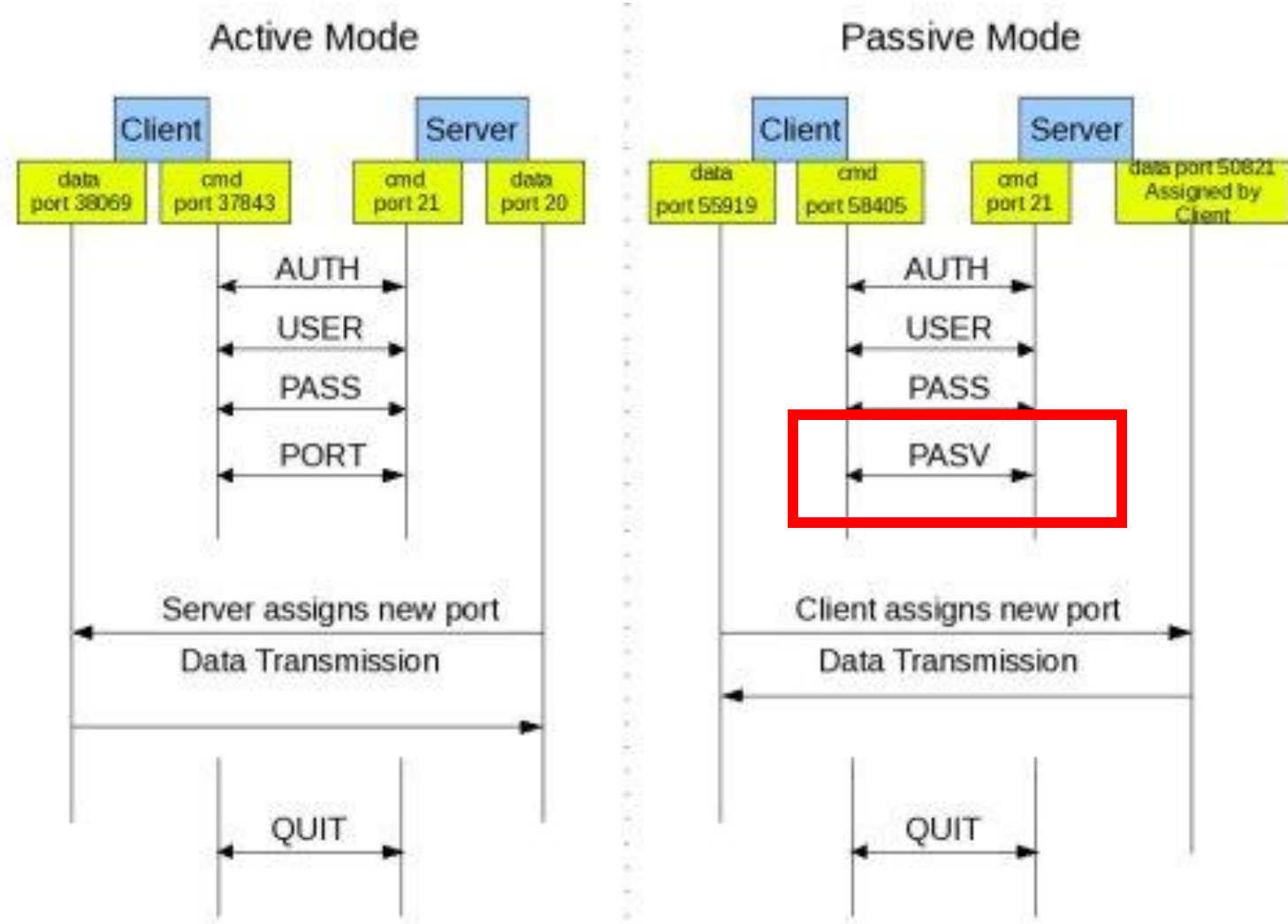


Image Source: <http://henrydu.com/blog/how-to/ftp-active-mode-vs-passive-mode-106.html>

FTP Data Transfer Modes

- **Stream mode:** Data is sent as a continuous stream, relieving FTP from doing any processing. Rather, all processing is left up to TCP. No End-of-file indicator is needed, unless the data is divided into records.
- **Block mode:** FTP breaks the data into several blocks (block header, byte count, and data field) and then passes it on to TCP.
- **Compressed mode:** Data is compressed using a simple algorithm (usually run-length encoding).

Source: Wikipedia

FTP Sample Commands and Response Codes

Sample commands:

- Sent as ASCII text over control channel
- USER *username*
- PASS *password*
- LIST: Return list of file in current directory
- RETR *filename*: Retrieves (gets) file
- STOR *filename*: Stores (puts) file onto remote host

Sample return codes

- Status code and phrase (as in HTTP)
- 331 username OK, password required
- 125 data connection already open; transfer starting
- 425 can't open data connection
- 452 error writing file

Application Layer Interfacing



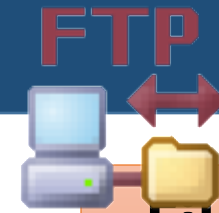
**Name Service
(DNS)**



**Web
(HTTP)**



**Email
(SMTP, POP, IMAP)**



**File Transfer
(FTP)**

**End to end
packet delivery**

UDP

**Connection
Establishment**

**Reliable Data
Delivery**

**Flow and
Congestion
Control**

**Ordered Packet
Delivery**

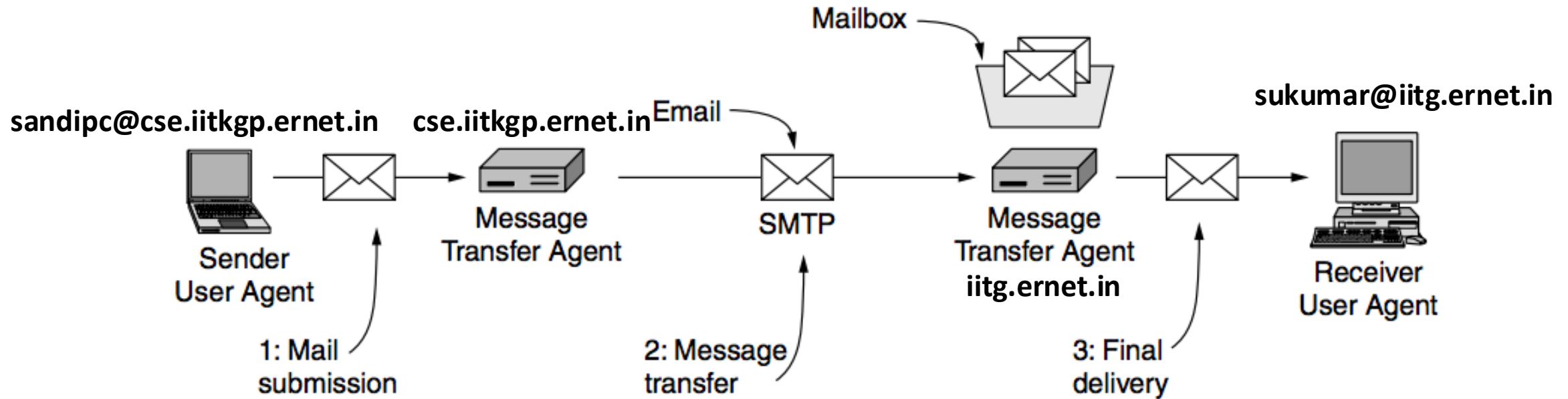
Transport

TCP

Network

Data Link

Electronic Mails – Architecture and Services



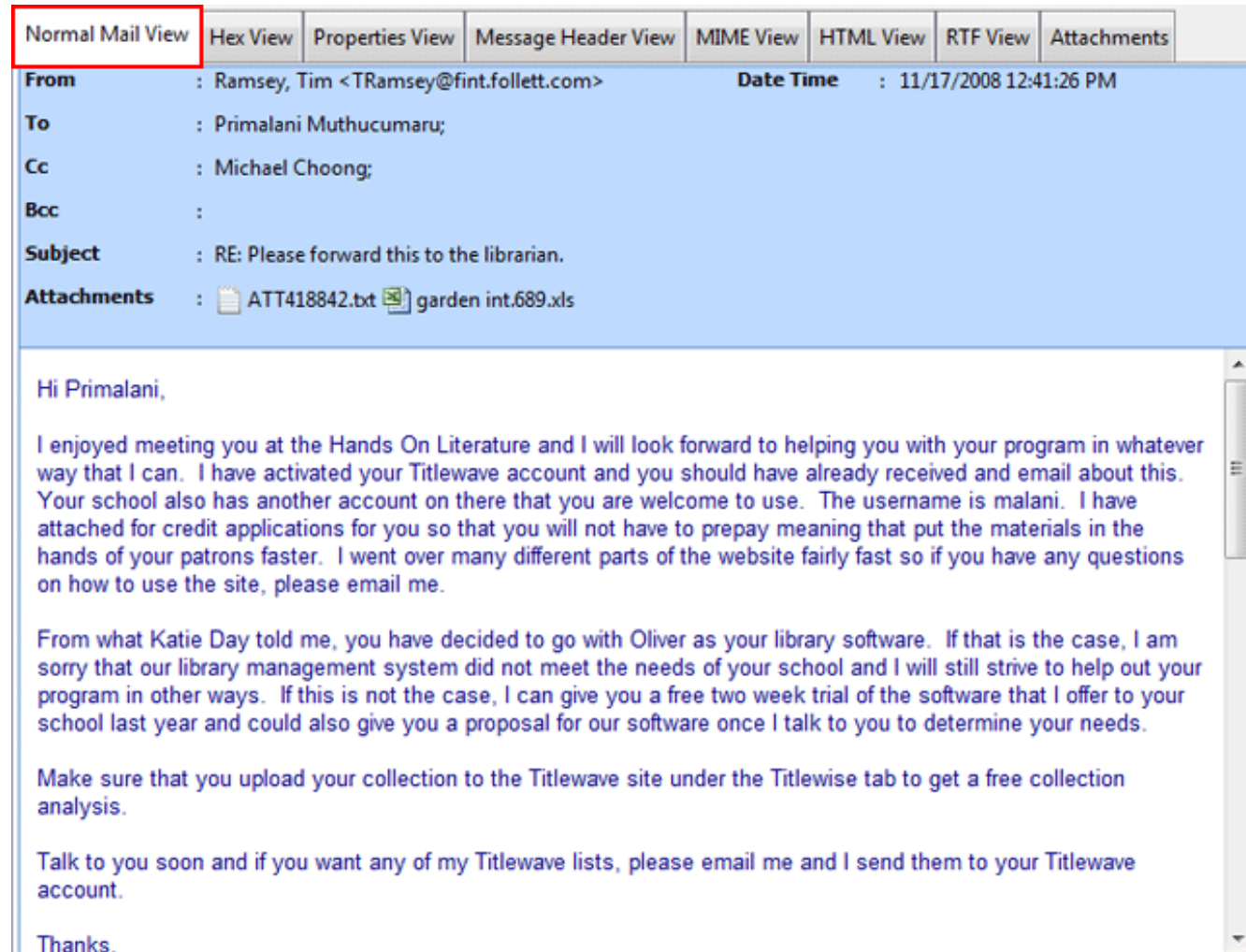
- **User Agent:** Allow people to read and send emails.
- **Message Transfer Agents (main servers):** Move the message from the source to the destination

Message Transfer Agents

- System processes run in the background on mail servers (always available).
- Automatically move emails through the system from the originator to the recipient
- Uses **Simple Mail Transfer Protocol (SMTP)** – RFC 821, RFC 5321
- Implements **mailing lists**, an identical copy of message is delivered to everyone in the list (btech@iitkgp.ac.in)
- Implements **Mailboxes**, to store all the emails received for a user

Message Format (RFC 5322)

- An envelope containing message header and message body



The Internet Message Format (RFC 5322)

- **Header fields (for message transport):**

Header	Meaning
To:	E-mail address(es) of primary recipient(s)
Cc:	E-mail address(es) of secondary recipient(s)
Bcc:	E-mail address(es) for blind carbon copies
From:	Person or people who created the message
Sender:	E-mail address of the actual sender
Received:	Line added by each transfer agent along the route
Return-Path:	Can be used to identify a path back to the sender

The Internet Message Format (RFC 5322)

- Header fields (additional fields for message description):

Header	Meaning
Date:	The date and time the message was sent
Reply-To:	Email address to which replies should be sent
Message-Id:	Unique number for referencing this message later
In-Reply-To:	Message-Id of the message to which this is a reply
References:	Other relevant Message-Ids
Keywords:	User-chosen keywords
Subject:	Short summary of the message for the one-line display

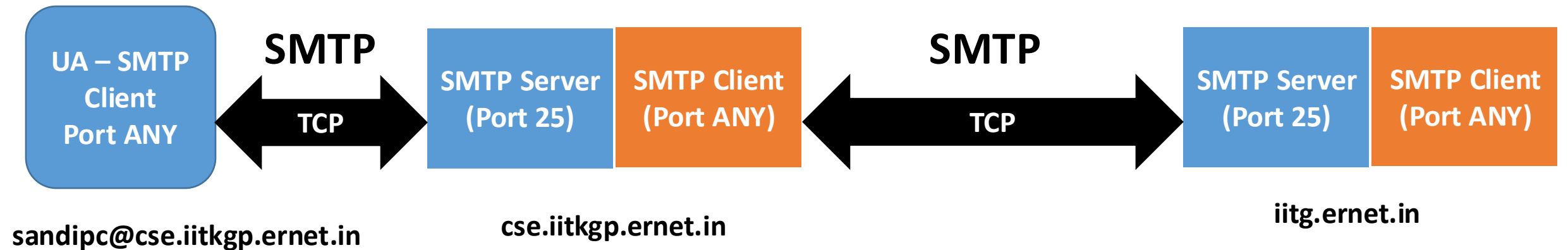
MIME – The Multipurpose Internet Mail Extension

- ARPANET: email consisted exclusively of text messages written in English and expressed in ASCII
- MIME: Use multi-language and multimedia contents (audio, image etc.) inside an email.
- **Additional message headers for MIME:**

Header	Meaning
MIME-Version:	Identifies the MIME version
Content-Description:	Human-readable string telling what is in the message
Content-Id:	Unique identifier
Content-Transfer-Encoding:	How the body is wrapped for transmission
Content-Type:	Type and format of the content

Message Transfer

- Uses SMTP Protocol
- Email is delivered by having the sending computer establishing TCP connection to port 25 of the receiving computer.



Message Transfer (SMTP)

Source: Computer Networks
(5th Edition) by Tanenbaum,
Wetherell

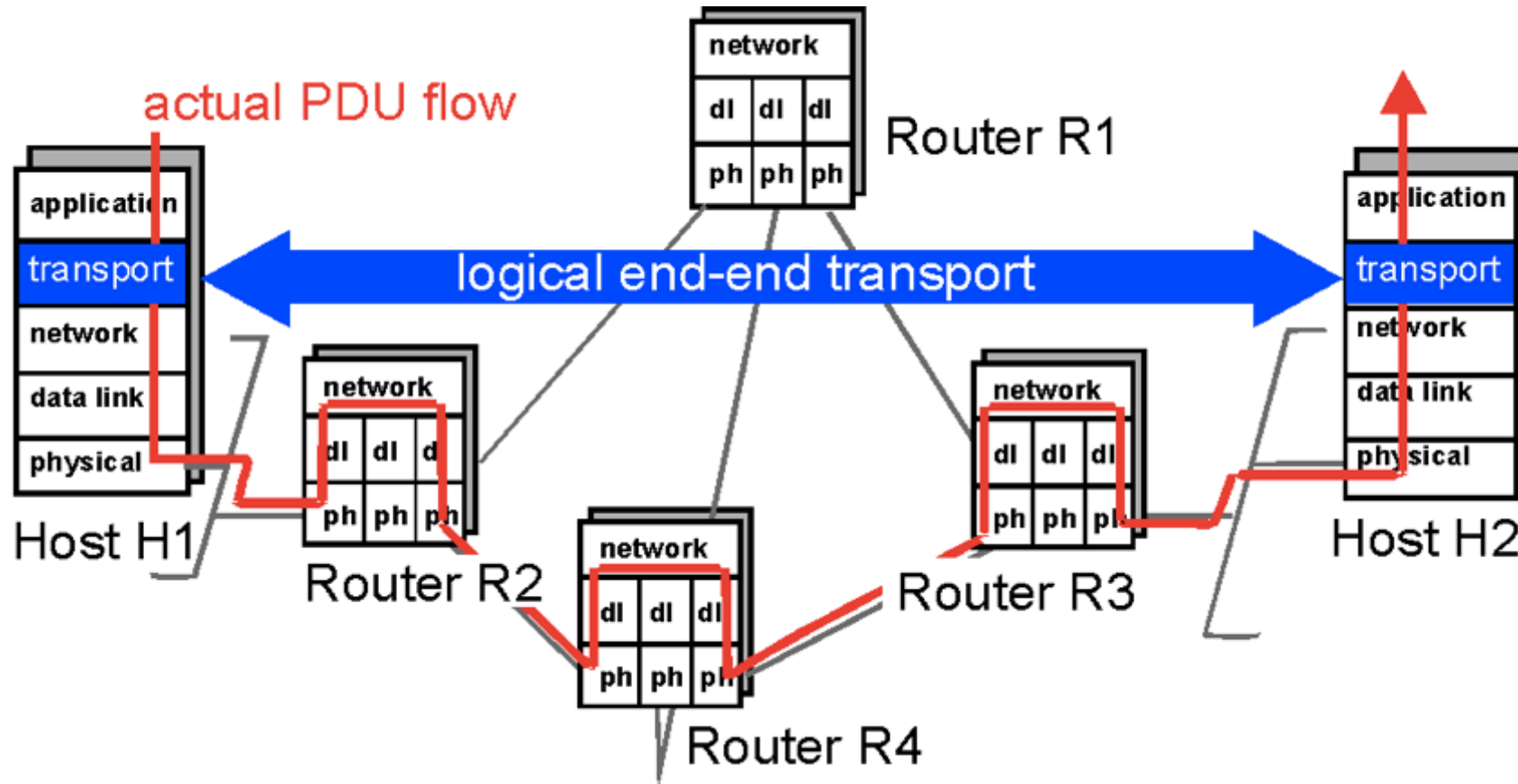
```
S: 220 ee.uwa.edu.au SMTP service ready
C: HELO abcd.com
S: 250 cs.washington.edu says hello to ee.uwa.edu.au
C: MAIL FROM: <alice@cs.washington.edu>
S: 250 sender ok
C: RCPT TO: <bob@ee.uwa.edu.au>
S: 250 recipient ok
C: DATA
S: 354 Send mail; end with "." on a line by itself
C: From: alice@cs.washington.edu
C: To: bob@ee.uwa.edu.au
C: MIME-Version: 1.0
C: Message-Id: <0704760941.AA00747@ee.uwa.edu.au>
C: Content-Type: multipart/alternative; boundary=qwertyuiopasdfghjklzxcvbnm
C: Subject: Earth orbits sun integral number of times
C:
C: This is the preamble. The user agent ignores it. Have a nice day.
C:
C: --qwertyuiopasdfghjklzxcvbnm
C: Content-Type: text/html
C:
C: <p>Happy birthday to you
C: Happy birthday to you
C: Happy birthday dear <bold> Bob </bold>
C: Happy birthday to you
C:
C: --qwertyuiopasdfghjklzxcvbnm
C: Content-Type: message/external-body;
C:   access-type="anon-ftp";
C:   site="bicycle.cs.washington.edu";
C:   directory="pub";
C:   name="birthday.snd"
C:
C: content-type: audio/basic
C: content-transfer-encoding: base64
C: --qwertyuiopasdfghjklzxcvbnm
C: .
S: 250 message accepted
C: QUIT
S: 221 ee.uwa.edu.au closing connection
```

Final Delivery

- Pull type protocol – UA at the receiver side pulls the emails from mail server after login.
- **Post Office Protocol, Version 3 (POP3)** – an earlier protocol for email delivery
- **Internet Message Access Protocol, Version 4 (IMAP v4)** – RFC 3501
 - The email server runs an IMAP server at port 143
 - The user agent runs IMAP client
 - The client connects to the server and issues mail delivery commands

IMAP (Version 4) Commands

Command	Description
CAPABILITY	List server capabilities
STARTTLS	Start secure transport (TLS; see Chap. 8)
LOGIN	Log on to server
AUTHENTICATE	Log on with other method
SELECT	Select a folder
EXAMINE	Select a read-only folder
CREATE	Create a folder
DELETE	Delete a folder
RENAME	Rename a folder
SUBSCRIBE	Add folder to active set
UNSUBSCRIBE	Remove folder from active set
LIST	List the available folders
LSUB	List the active folders
STATUS	Get the status of a folder
APPEND	Add a message to a folder
CHECK	Get a checkpoint of a folder
FETCH	Get messages from a folder
SEARCH	Find messages in a folder
STORE	Alter message flags
COPY	Make a copy of a message in a folder
EXPUNGE	Remove messages flagged for deletion
UID	Issue commands using unique identifiers
NOOP	Do nothing
CLOSE	Remove flagged messages and close folder
LOGOUT	Log out and close connection



- Next, we'll go for the Transport Layer